Module 6: Developing REST APIs

AWS Academy Cloud Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 1: Introduction

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module objectives

At the end of this module, you should be able to do the following:

- Recognize APIs
- Describe Amazon API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module overview

Sections

1. Introduction
2. Introducing APIs
3. Introducing API Gateway
4. Creating a REST API
5. Integrating with API Gateway
6. Deploying an API
7. Controlling access to a REST API
8. Monitoring a REST API
9. Optimizing API Gateway

Lab

Developing REST APIs with Amazon API Gateway

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

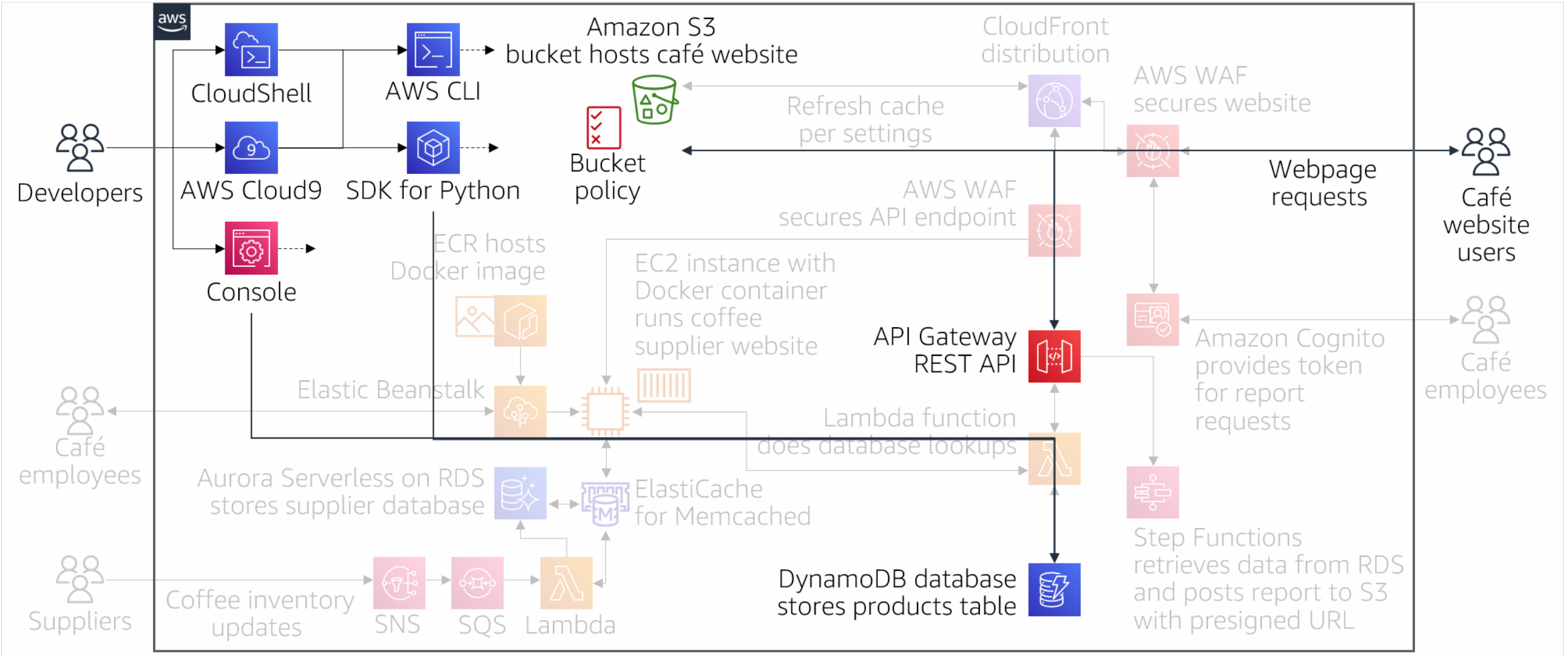
Café business requirement

Product information for the café is now being stored in a database, and Sofía wants to use an API to integrate the website with the database. She wants to test the front end and get approval from Frank and Martha before completing the full integration.

```
cursor.close()

sql = "INSERT INTO hive (dev_id, humidity, temperature) VALUES (%s, %s, %s)"

def on_connect(client, userdata, flags, rc):
    print("Connected with result code " + str(rc))

    client.subscribe("+/devices/+/up")

def on_message(client, userdata, msg):
    print(msg.topic + " &&& " + str(msg.payload)+"\n")
    data = json.loads(msg.payload.decode('utf8').replace("'", '"'))
    val = (data["dev_id"], data["payload_fields"]["humidity"], data["payload_fields"]["temperature"])
    connection = mysql.connector.connect(
        host="bees-mariaDB",
        user="bees",
        passwd=mysql_password,
        database="bees"
    )
    cursor = connection.cursor()
    cursor.execute(sql, val)
    connection.commit()
    print(mycursor.rowcount, "record inserted.")
    cursor.close()
    connection.close()

client = mqtt.Client()
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

REST APIs as part of developing a cloud application



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2: Introducing APIs

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What is an API?

An API is a software mechanism that simplifies development by doing the following:

Abstracting implementation details

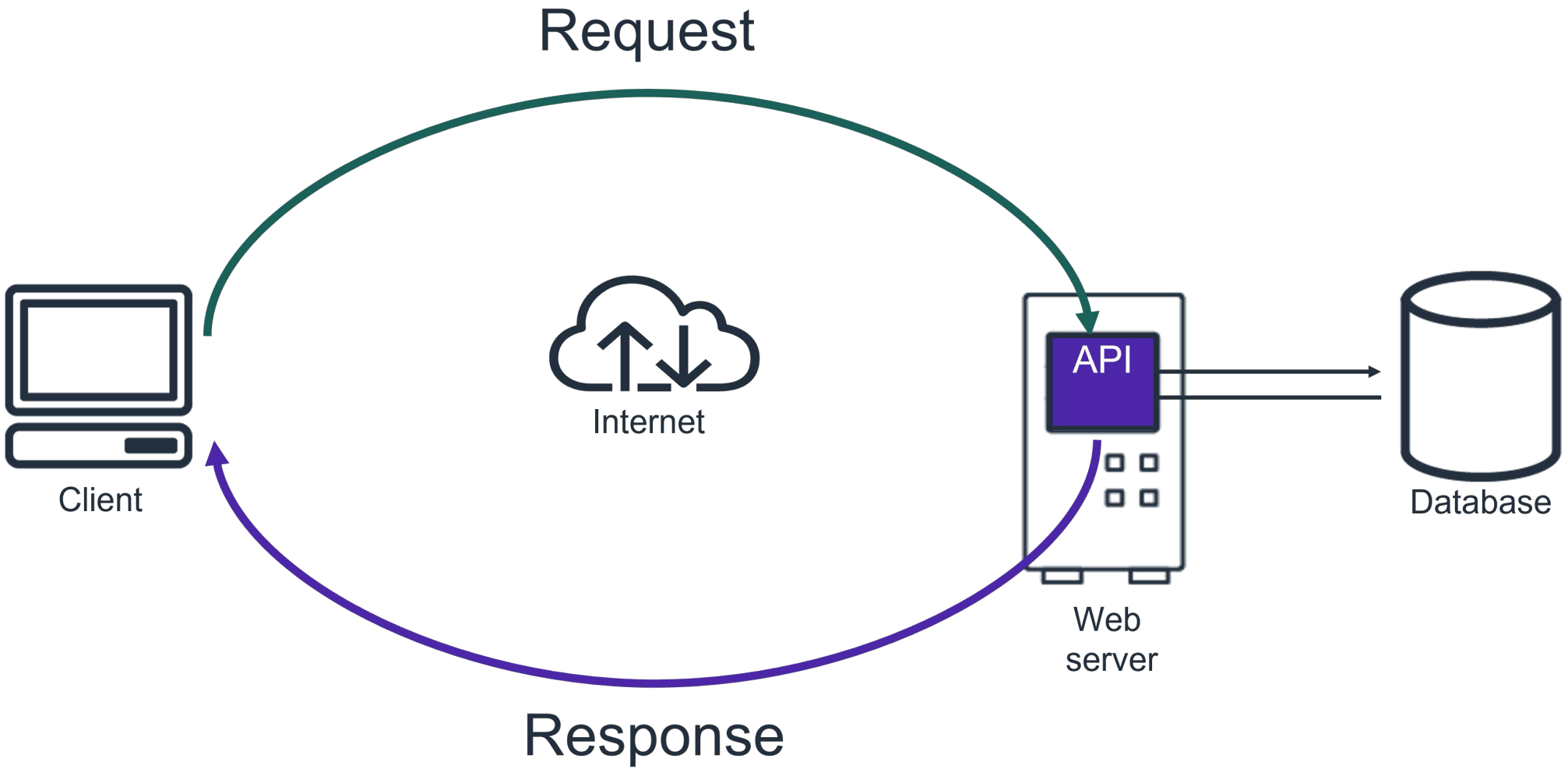
Exposing only objects or actions that a developer needs

Establishing how an information provider and an information user communicate

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

A typical application

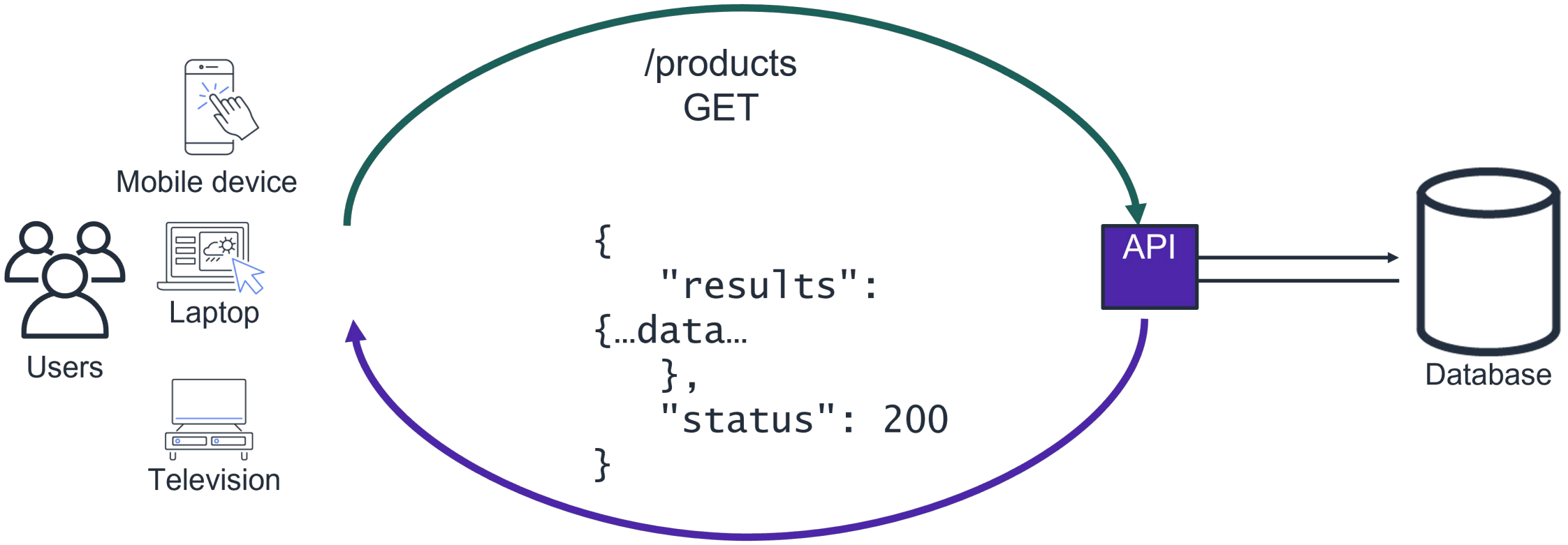


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Requests and responses

Request

Uses a standard protocol (SOAP, WebSockets, HTTP, HTTPS)



Response

Includes data and status codes

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What is a RESTful API?

Representational State Transfer (REST)

- Architectural style
- Standard way of structuring requests from a client to a server

RESTful API

- An API that adheres to REST principles

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2 key takeaways

- APIs simplify development by abstracting implementation details and providing a shared definition of how to communicate with an application.
- Clients send requests to an API and get a response back.
- REST is a standard way to structure API requests.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3: Introducing API Gateway

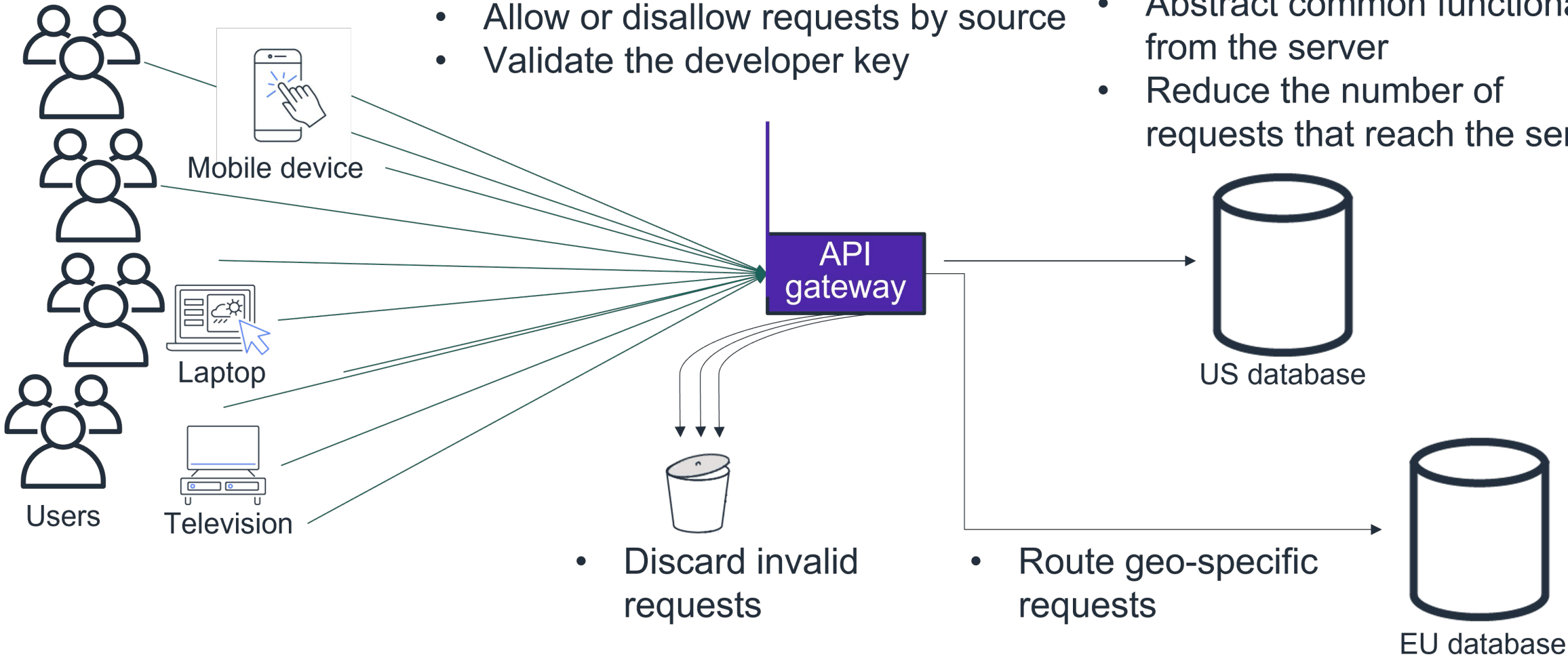
Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

An API gateway is a proxy

- Limit the request rate
- Allow or disallow requests by source
- Validate the developer key

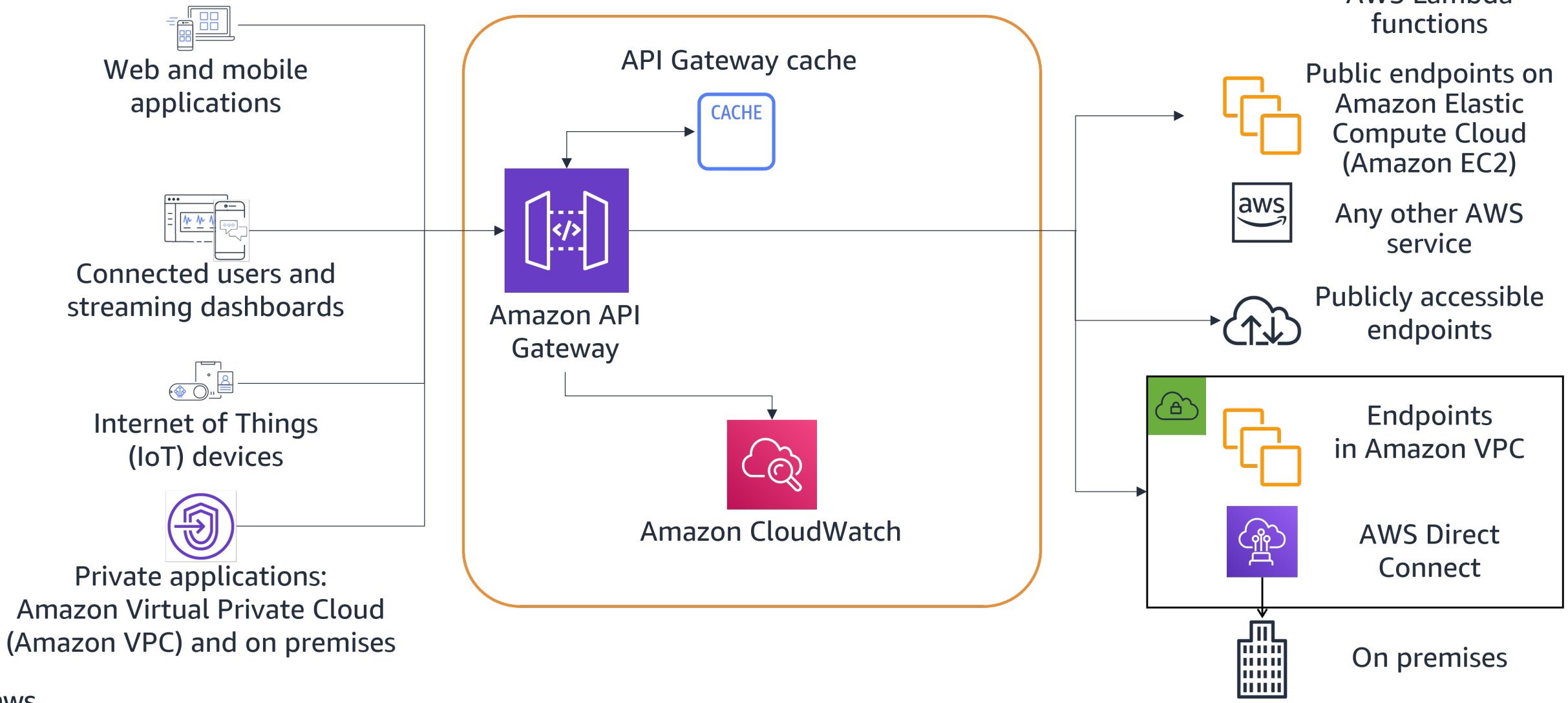
- Abstract common functionality from the server
- Reduce the number of requests that reach the server



- Discard invalid requests
- Route geo-specific requests

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What is Amazon API Gateway?



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

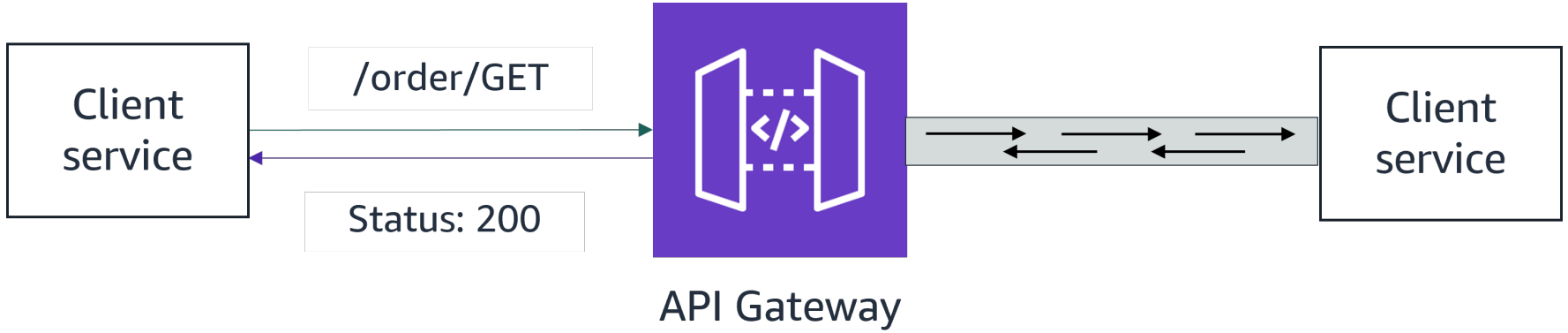
API Gateway supported protocols

RESTful

- Request/response
- HTTP methods (GET, POST)
- Short-lived communication
- Stateless

WebSockets

- Two-way communication channel
- Long-lived communication
- Stateful



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

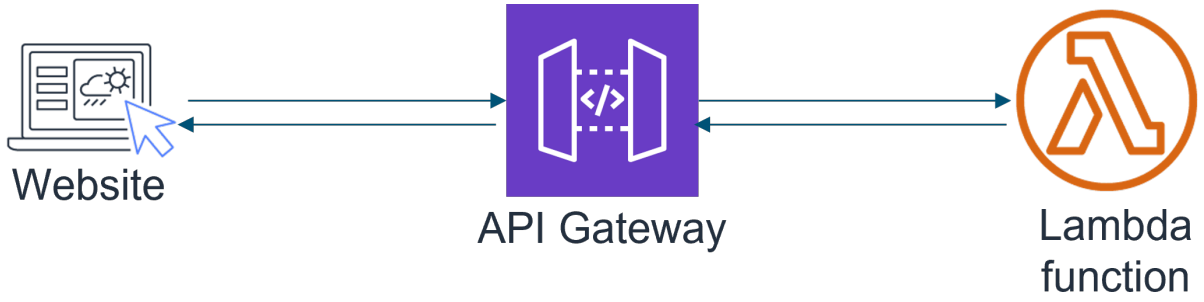
RESTful APIs in API Gateway

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Two types of RESTful APIs in API Gateway

- REST API
- Gives the developer full control over API requests and responses
- Supports features that are not yet available in HTTP APIs

- HTTP API
- Simplifies the development of APIs that require only API proxy functionality
- Designed for the lowest cost and lowest latency



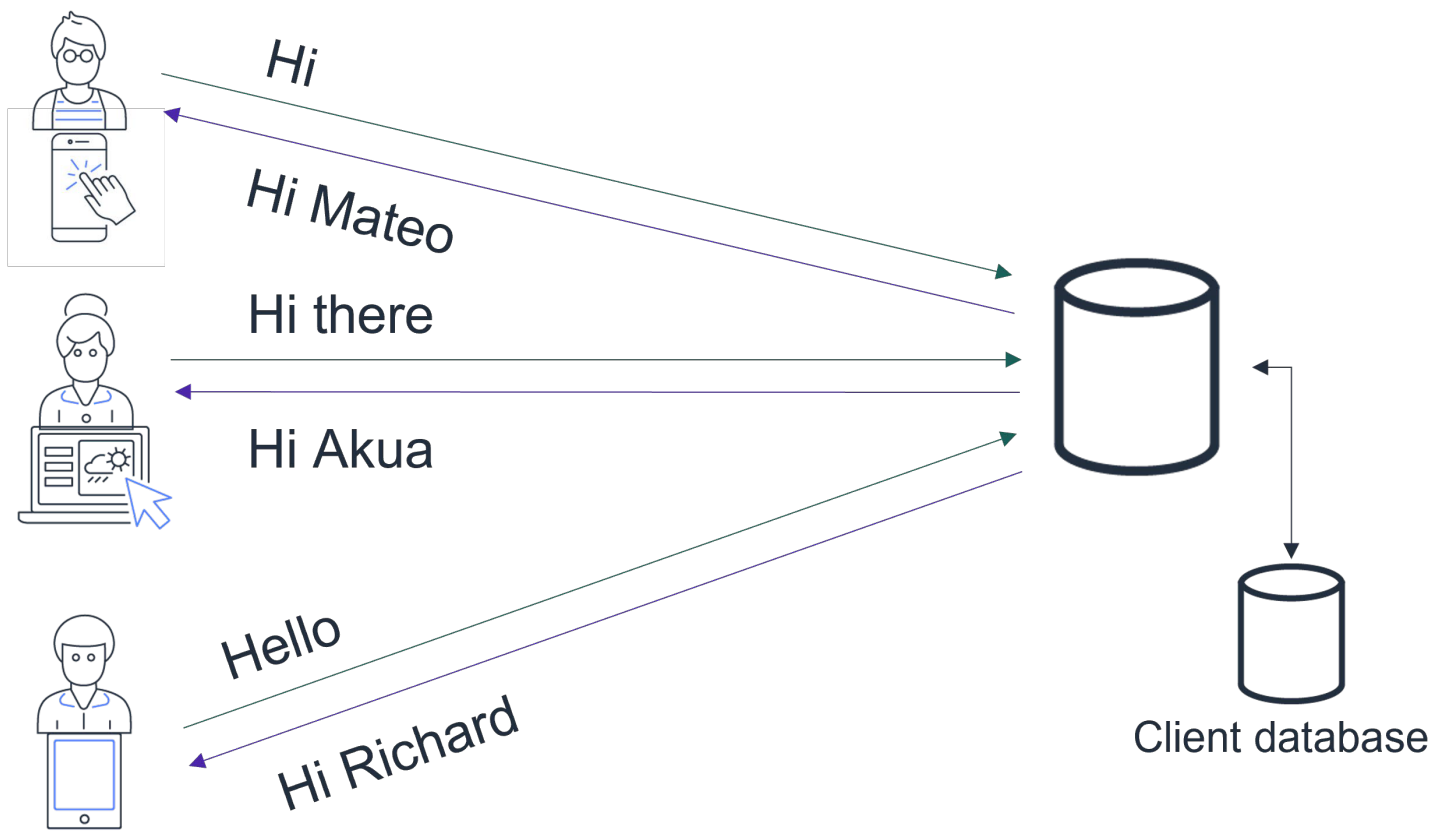
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

WebSocket APIs in API Gateway

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

WebSockets are bidirectional

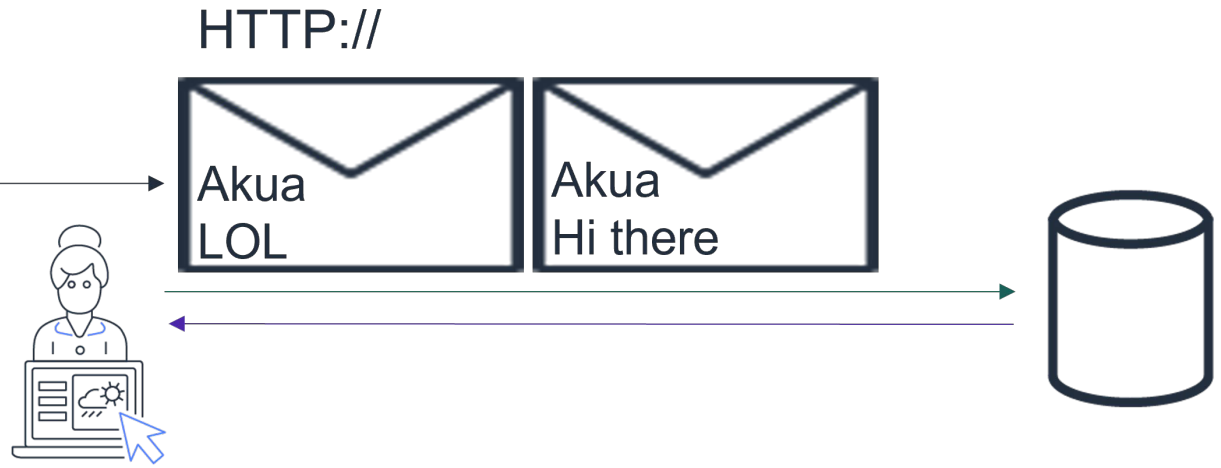
Real-time, two-way communication



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

More efficient for chatty applications

Envelope bloat per message



Just the data after the connection is established

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

API Gateway management

Develop

AWS Management Console

AWS Command Line Interface (AWS CLI)

OpenAPI Specification (Swagger)

Deploy

AWS Serverless Application Model (AWS SAM)

AWS CloudFormation

AWS Cloud Development Kit (AWS CDK)

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3 key takeaways

- Amazon API Gateway is a fully managed service that provides API proxy and simplifies API development.
- API Gateway supports RESTful and WebSocket APIs.
- API Gateway integration with AWS services frees developers from writing integration code.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4: Creating a REST API

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating a REST or HTTP API

Create an API in API Gateway:

- Use the AWS CLI
- Use the API Gateway console

Base structure for REST and HTTP APIs

```
https://{restapi_id}.execute-api.{region}.amazonaws.com/{stage_name}/
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example: Creating an HTTP API that is connected to a Lambda function

With the AWS CLI

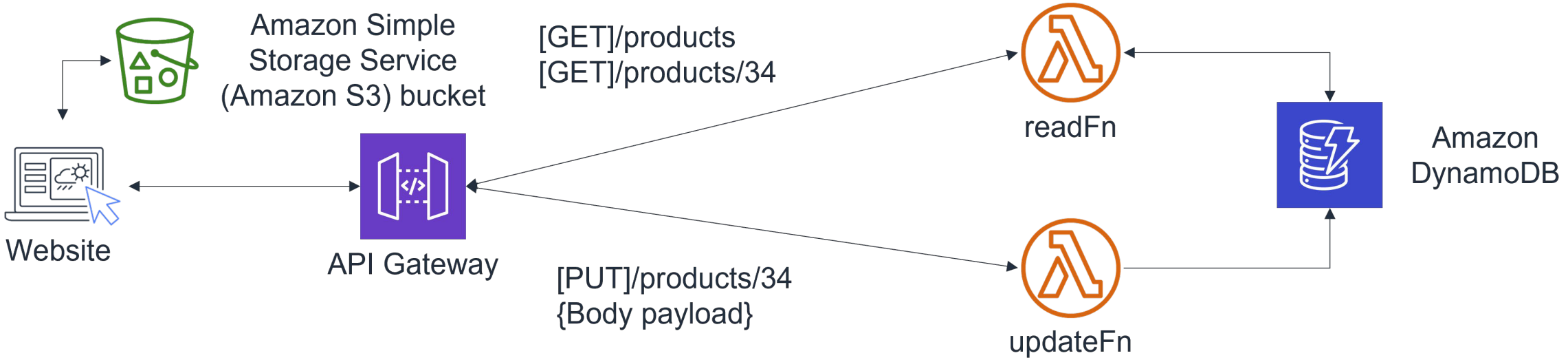
```
aws apigatewayv2 create-api \
                --name my-api \
                --protocol-type HTTP \
                --target
arn:aws:lambda:us-east-
2:123456789012:function:function-name
```

With the console

1. Open the API Gateway console.
2. Choose Build on the HTTP panel.
   - If any APIs already exist, the console displays a list of existing APIs and an option to create an API. Choose Create API. Then choose Build on the HTTP API panel.
3. Choose Add integration, and choose a Lambda function in the same Region as the API that you are creating.
4. Enter a name for your API.
5. Choose Review and Create.
6. Choose Create.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Methods and routes




```
#To get all products
[GET] https://api-id.execute-api.us-east-2.amazonaws.com/products

#To get a specific product by ID (34)
[GET] https://api-id.execute-api.us-east-2.amazonaws.com/products/34

#To update (replace) a specific product by ID (34)
[PUT] https://api-id.execute-api.us-east-2.amazonaws.com/products/34
BODY #some keys and values
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating GET routes

```
aws apigatewayv2 create-route \
            --api-id 03z50l670a \
            --route-key 'GET /products/{product_id}' \
            --target arn:aws:lambda:us-east-2:123456789012:function:readFn

#and for the "all" products
aws apigatewayv2 create-route \
            --api-id 03z50l670a \
            --route-key 'GET /products' \
            --target arn:aws:lambda:us-east-2:123456789012:function:updateFn
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating a PUT route

```
aws apigatewayv2 create-route \
            --api-id 03z50l670a \
            --route-key 'PUT /product/{product_id}' \
            --target arn:aws:lambda:us-east-2:123456789012:function:updateFn
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using the greedy path variable {proxy+}

```
aws apigatewayv2 create-route \
            --api-id 03z50l670a \
            --route-key 'GET /products/{proxy+}' \
            --target arn:aws:lambda:us-east-2:123456789012:function:readFn
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating a WebSocket API

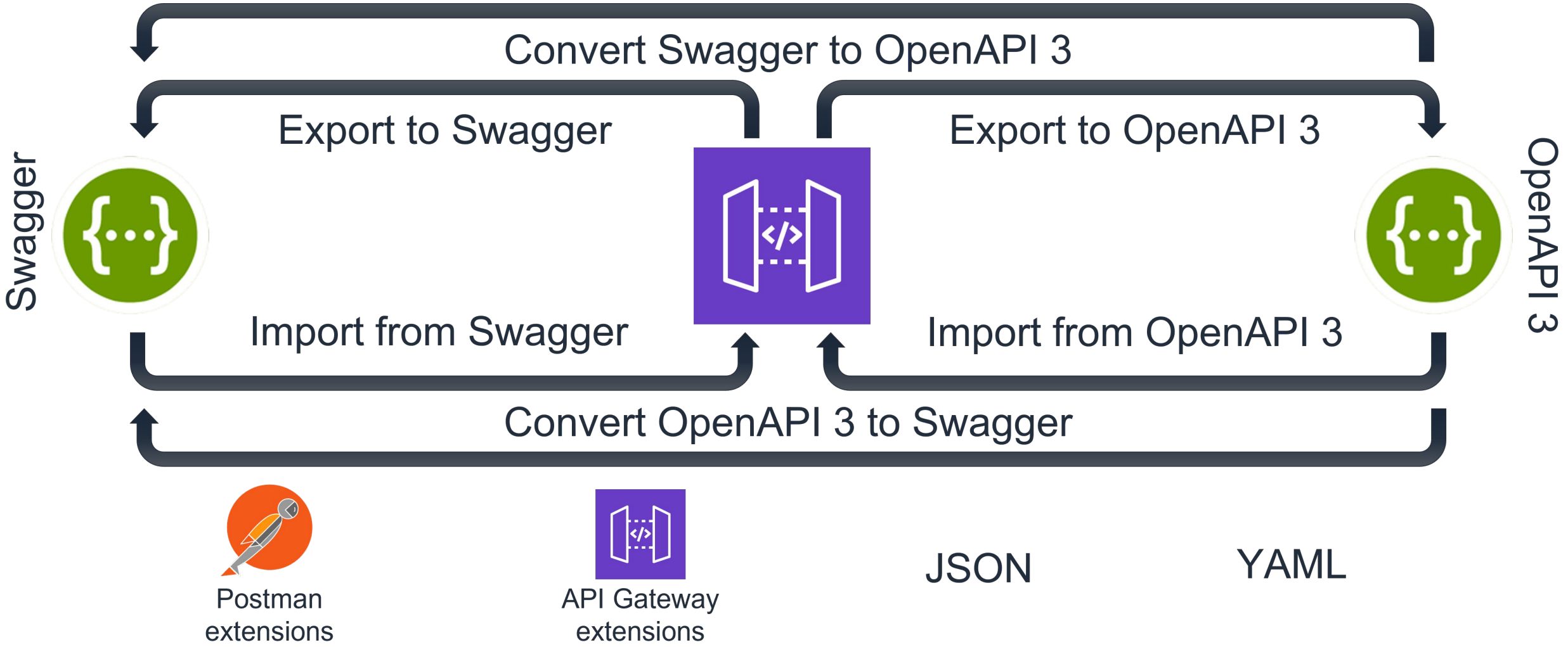
```
aws apigatewayv2 create-api \
    --name "myWebSocketApi" \
    --protocol-type WEBSOCKET \
    --route-selection-expression '$request.body.action'
```

JSON message

```
{
    "service" : "chat",
    "action" : "join",
    "data" : {
        "room" : "room1234"
    }
}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Importing an API



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4 key takeaways

- You can create APIs by:
  - Using the AWS CLI
  - Using the API Gateway console
  - Importing an OpenAPI definition
- As a best practice, use different paths to perform different actions and connect to different backend integrations.

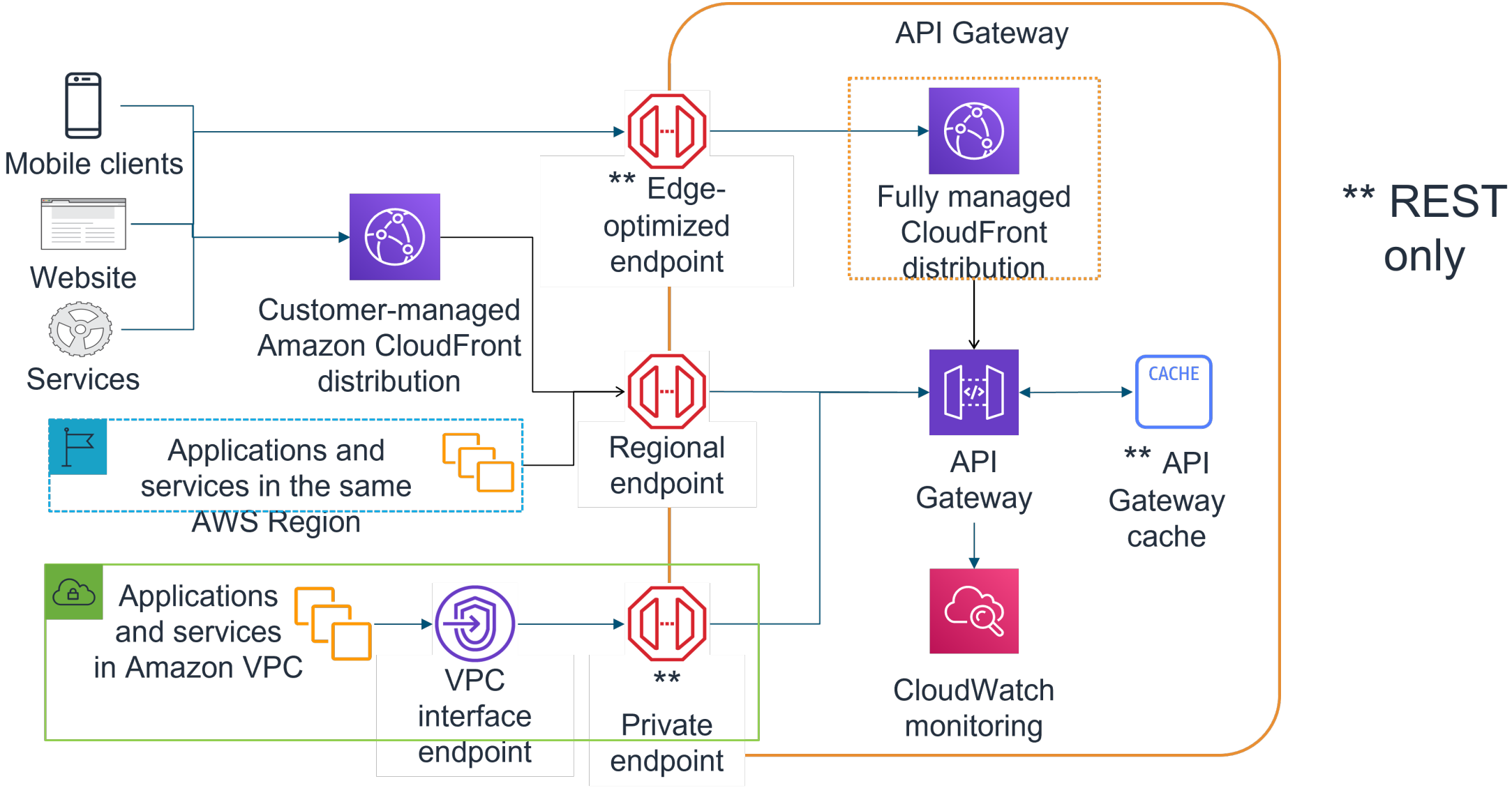
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5: Integrating with API Gateway

Module 6: Developing REST APIs

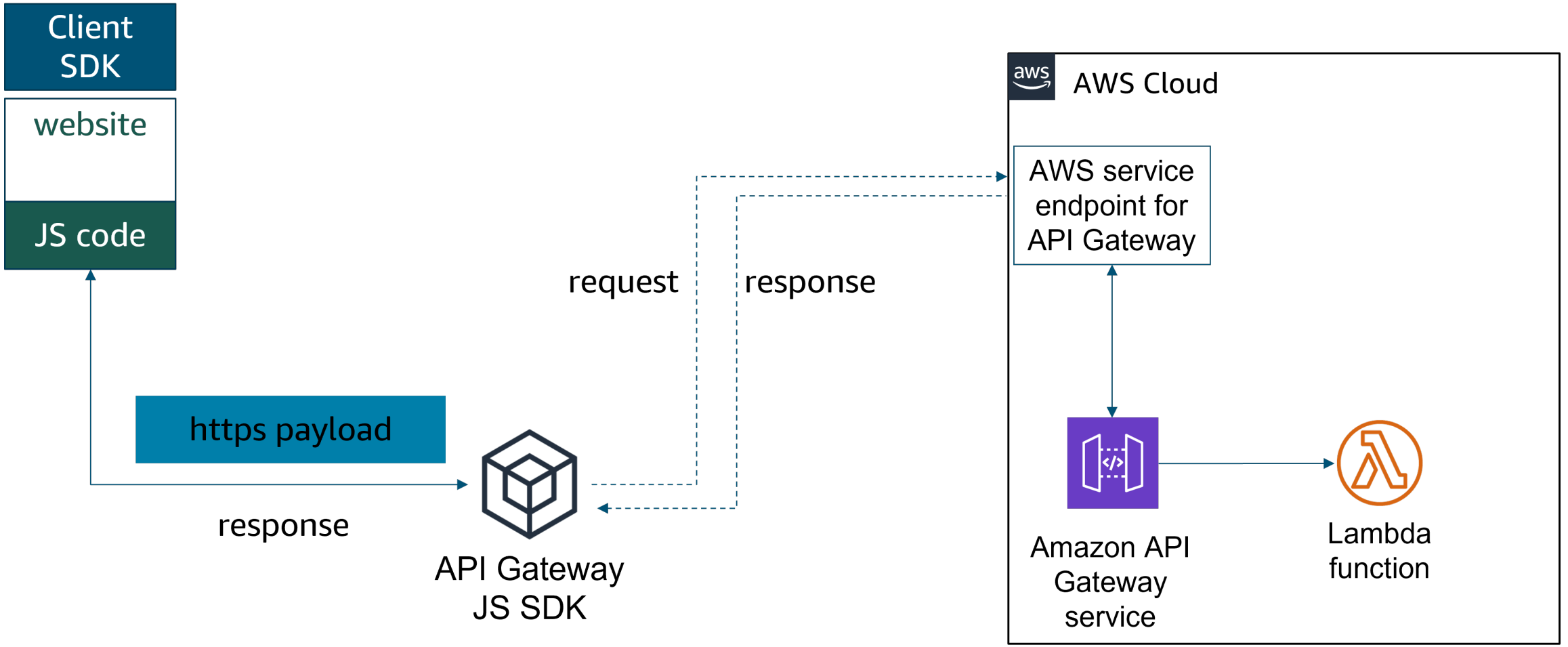
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Endpoint types



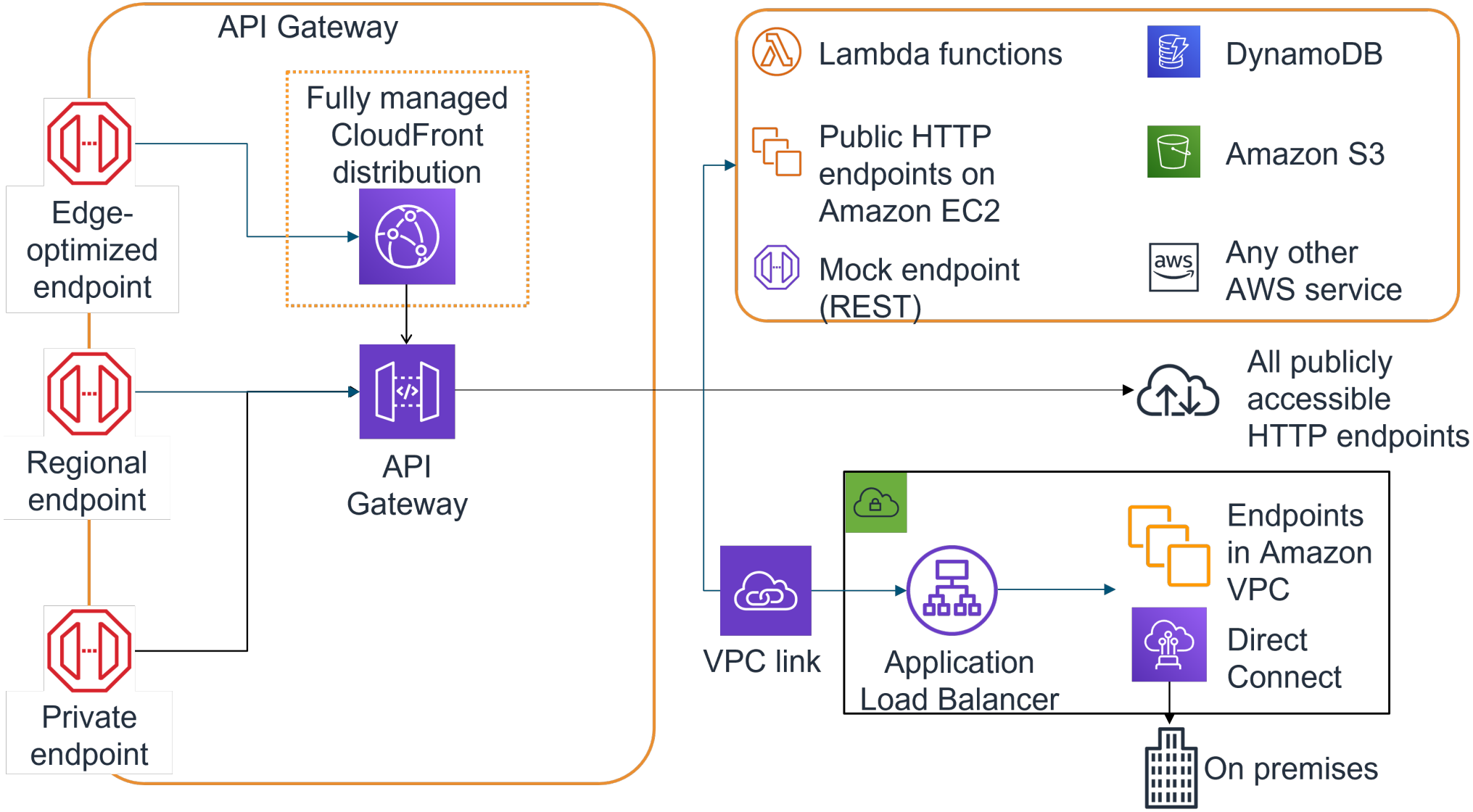
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using client-side SDKs



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Backend integrations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example integration flow

1. **Method request**
   - Modeling
   - Validation
   - Transformation

2. **Integration request**
   - Encapsulating HTTPs requests for the backend

Websites

Request

Response

Endpoint

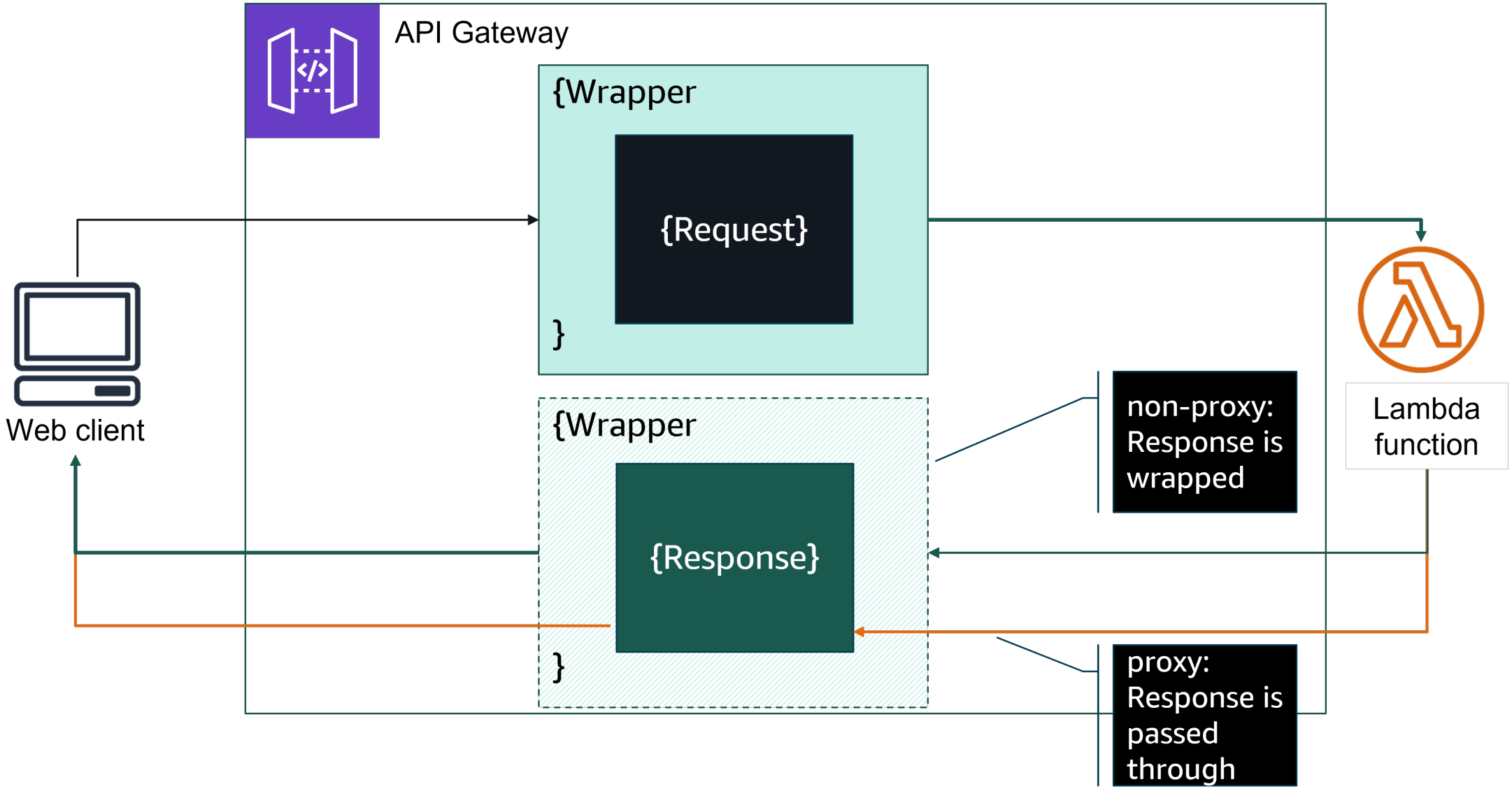
- DynamoDB
- Amazon S3
- Other AWS services
- Mock endpoint
- Lambda functions
- Public endpoints
- VPC
  - Endpoints in Amazon VPC

4. **Method response**
   - Modeling
   - Header validation

3. **Integration response**
   - Transformation
   - Custom errors

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lambda non-proxy vs. proxy integration



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example standard response in non-proxy integration

Lambda function response

```
{ "message": "Hello from Lambda!" }
```

API Gateway standard response in non-proxy integration

```
{
  "isBase64Encoded": false,
  "statusCode": 200,
  "body": "{ \"message\": \"Hello from Lambda!\" }",
  "headers": {
    "content-type": "application/json"
  }
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example Lambda error response in proxy integration

Lambda permissions error message

```
{ "message": "You are not allowed to see
this content" }
```

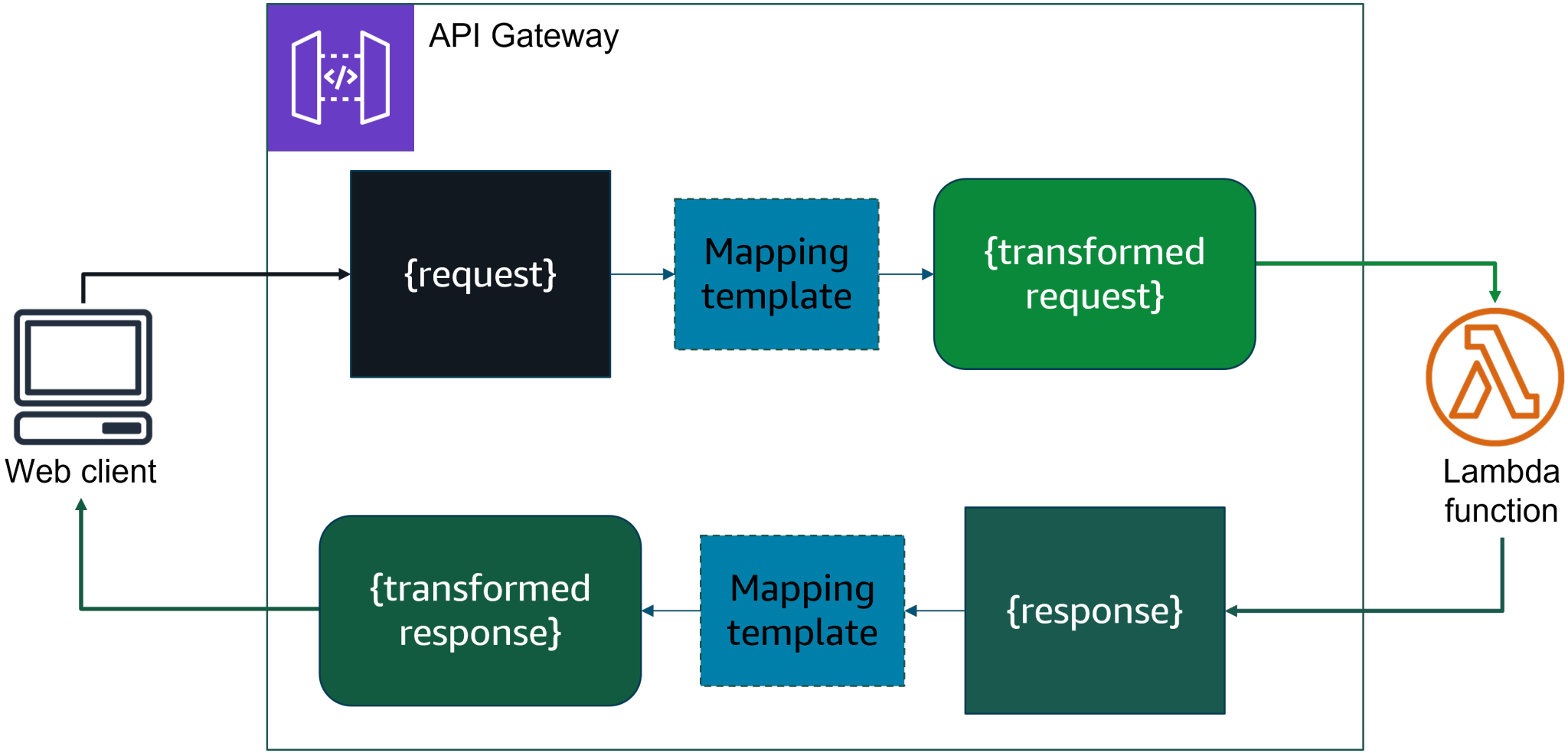
Lambda customized response in a proxy integration

```
{
    "cookies" : ["cookie1", "cookie2"],
    "isBase64Encoded": true,
    "statusCode": 403,
    "headers": { "headername": "headervalue", ... },
    "body": "You are not allowed to see this content"
}
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Modifying requests and responses in a non-proxy integration



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example: Modifying a request by using VTL

Before transformation

```
{
    "firstname": "John",
    "lastname": "Stiles",
    "phone": "555-555-5555"
    "city": "Miami",
    "state": "Florida",
    "favoritePizza": "pineapple"
}
```

After transformation

```
{
    "name": "John Stiles",
    "phone": "555-555-5555",
    "address":{
        "city": "Miami",
        "state": "Florida"
    }
    "favoritePizza": "Hawaiian"
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Transforming the response

Before VTL transformation

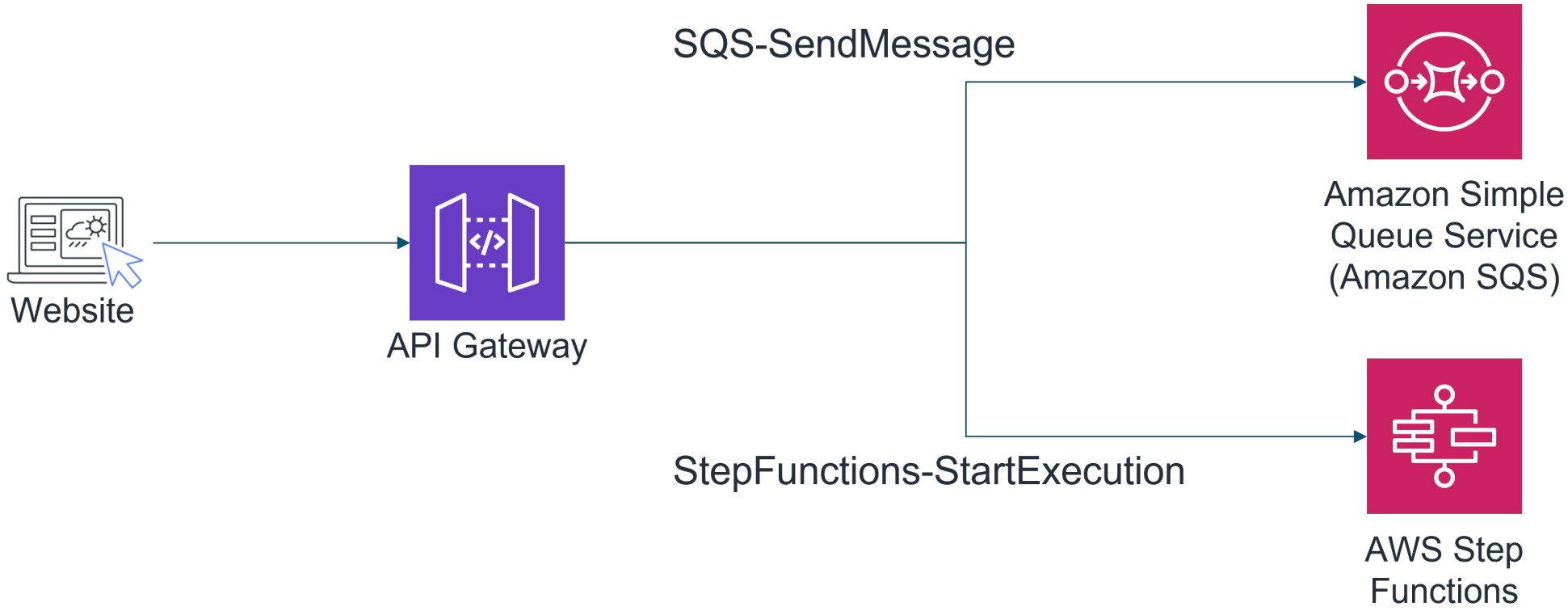
```
{
   "message": "Hello from Lambda",
}
```

After VTL transformation

```
{
    "cookies": […"],
    "isBase64Encoded": false,
    "statusCode": 200,
    "headers": [... ],
    "body": "Hello from Lambda"
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

First-class integrations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example: First-class integration

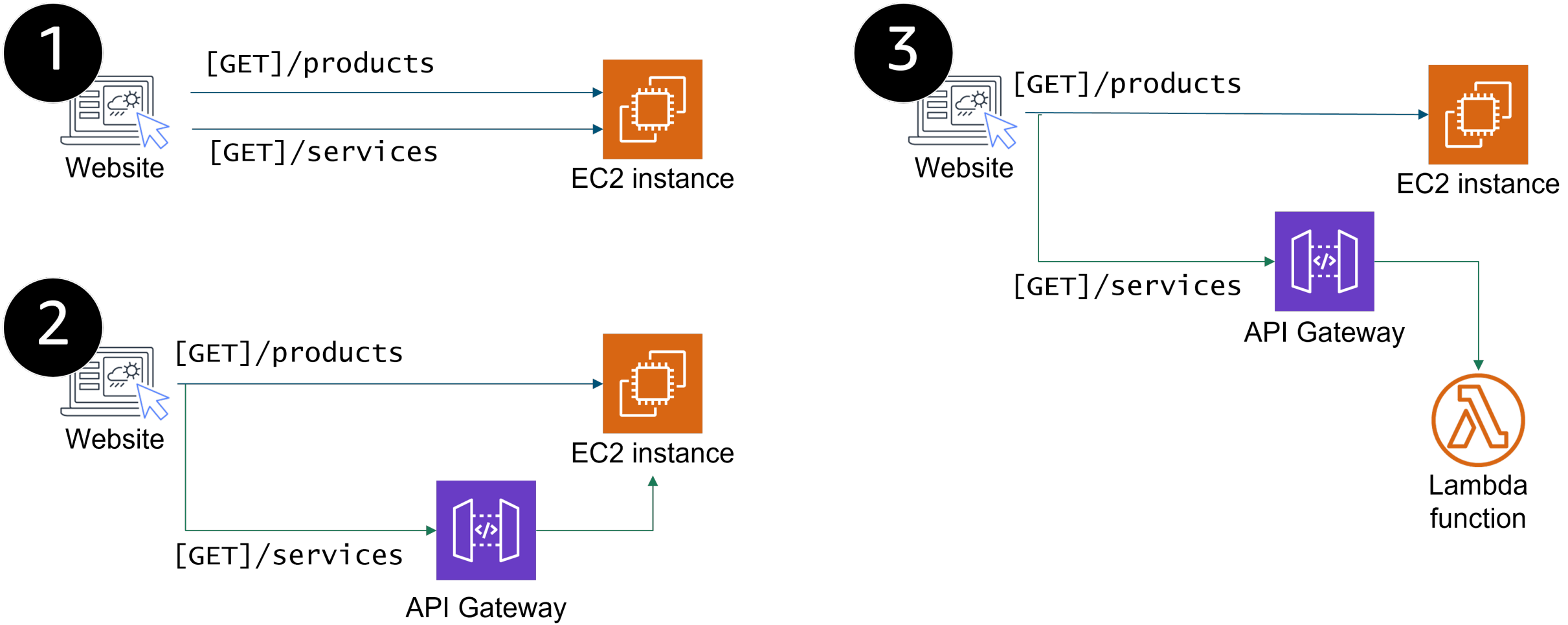
```
aws apigatewayv2 create-integration \
    --api-id abcdef123 \
    --integration-subtype SQS-SendMessage \
    --integration-type AWS_PROXY \
    --payload-format-version 1.0 \
    --credentials-arn arn:aws:iam::123456789012:role/apigateway-
sqs \
    --request-parameters '{"QueueUrl":
"$request.header.queueUrl", "MessageBody":
"$request.body.message"}'
```

Connect an HTTP API route to an AWS service API

You provide the mapping that the service requires

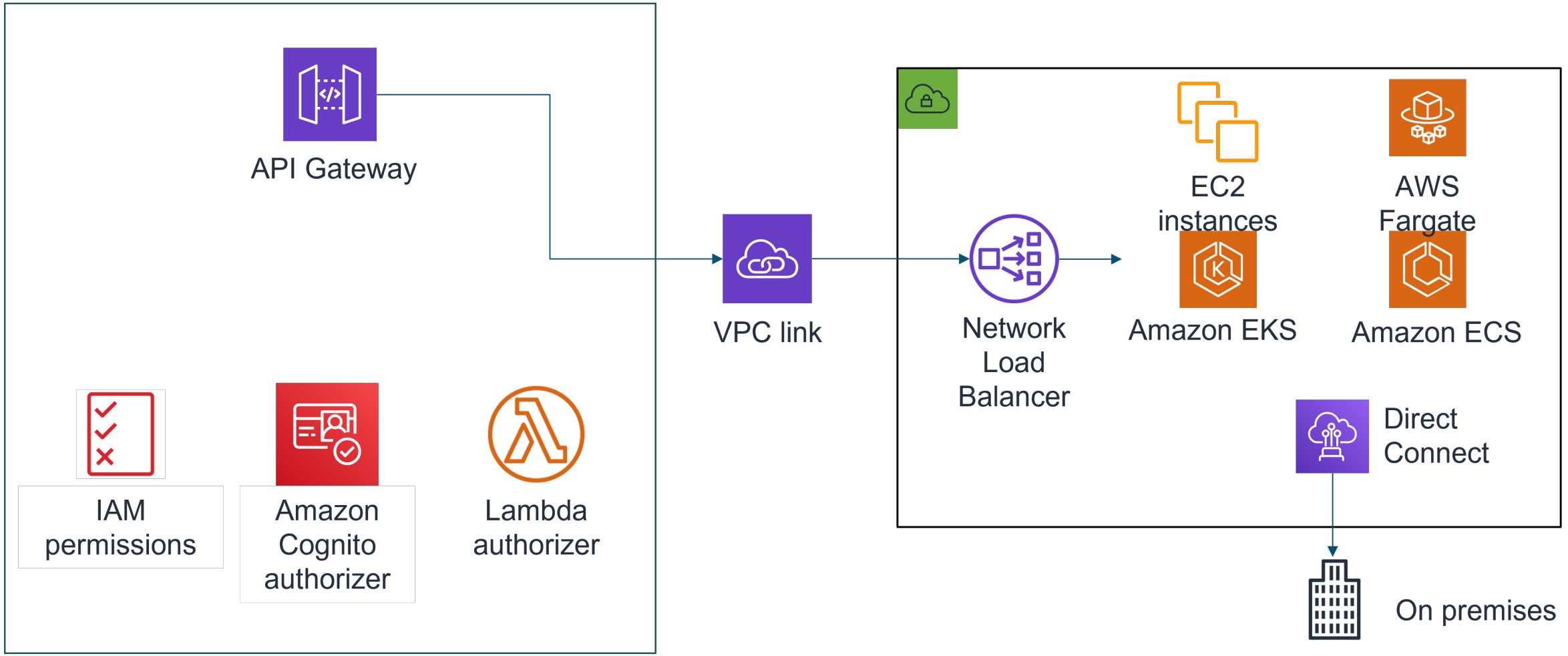
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

HTTP proxy integrations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Private integrations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example: Creating a private integration

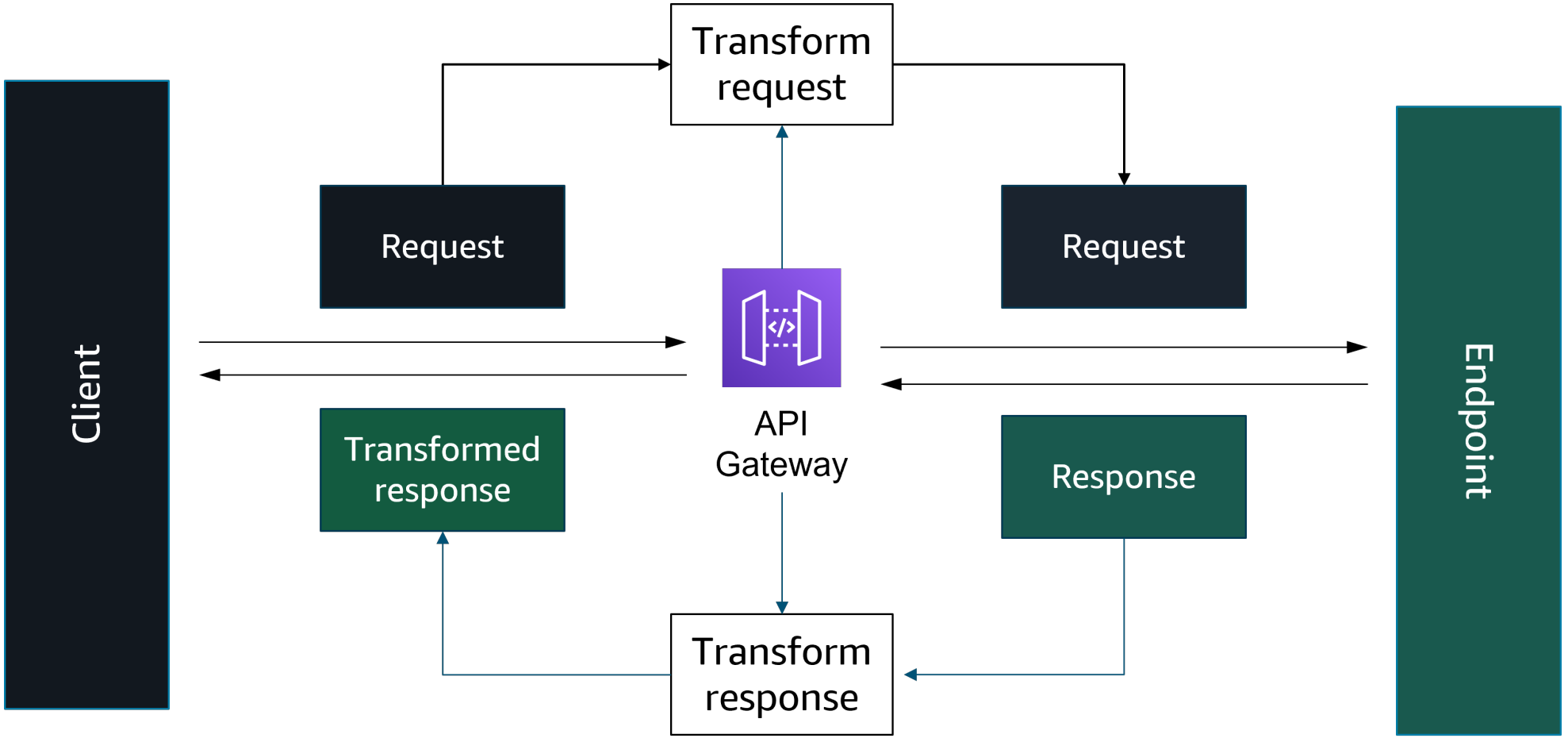
```
aws apigatewayv2 create-integration \
            --api-id api-id
    --integration-type HTTP_PROXY \
    --integration-method GET \
    --connection-type VPC_LINK \
    --connection-id VPC-link-ID \
    --integration-uri arn:aws:elasticloadbalancing:us-east-
2:123456789012:listener/app/my-load-
balancer/50dc6c495c0c9188/0467ef3c8400ae65
    --payload-format-version 1.0
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Transforming requests and responses

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Transforming requests and responses



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example request transformation

```
aws apigatewayv2 create-integration \
    --api-id abcdef123 \
    --integration-type HTTP_PROXY \
    --payload-format-version 1.0 \
    --integration-uri 'https://api.example.com' \
    --integration-method ANY \
    --request-parameters '{ "append:header.header1": "$context.requestId"
}'
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5 key takeaways

- API Gateway simplifies the work to integrate with AWS services from your client applications.
- Use Lambda proxy and HTTP proxy integrations for simple passthroughs to backend services.
- Use transformation options to modify requests and responses between clients and targets.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6: Deploying an API

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Deployments are associated to a stage

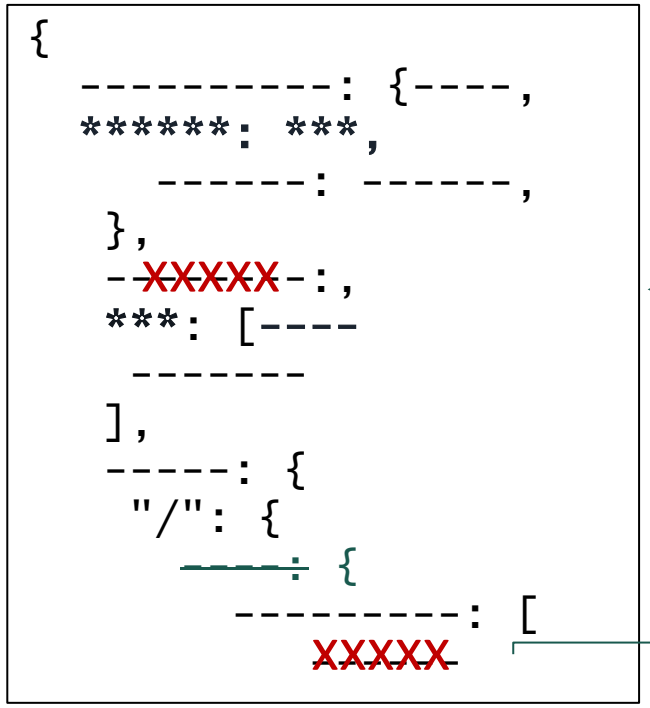
- Stages are snapshots of an API:
  - Named as you like
  - Identified by API ID and stage name
  - Included in the URL that is used to invoke the API
- Developers create multiple stages to:
  - Differentiate versions for development environments (like dev and prod)
  - Connect different stages to different backends by using stage variables
  - Optimize a particular deployment—for example, enabling caching or throttling per stage

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using stages as development environments

dev stage

Version 1.0.3: For your eyes only

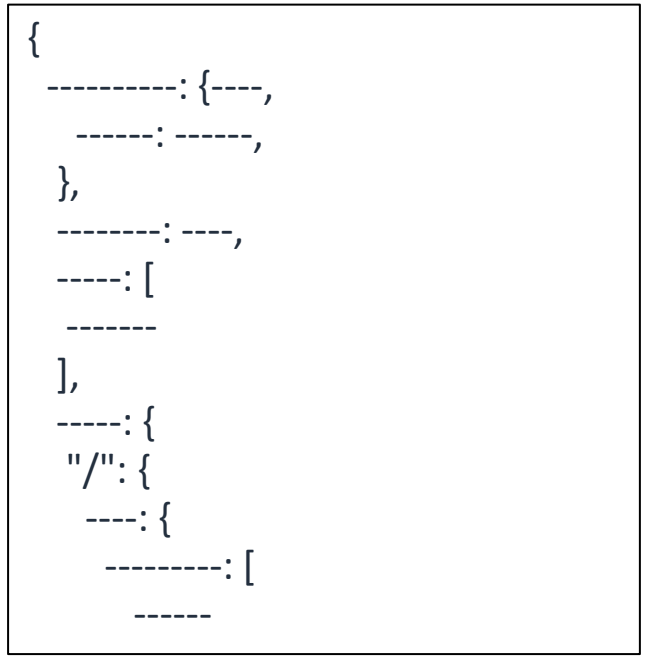


- Code updates
- Refactoring

https://api-id.execute-api.us-east-2.amazonaws.com/dev/products

prod stage

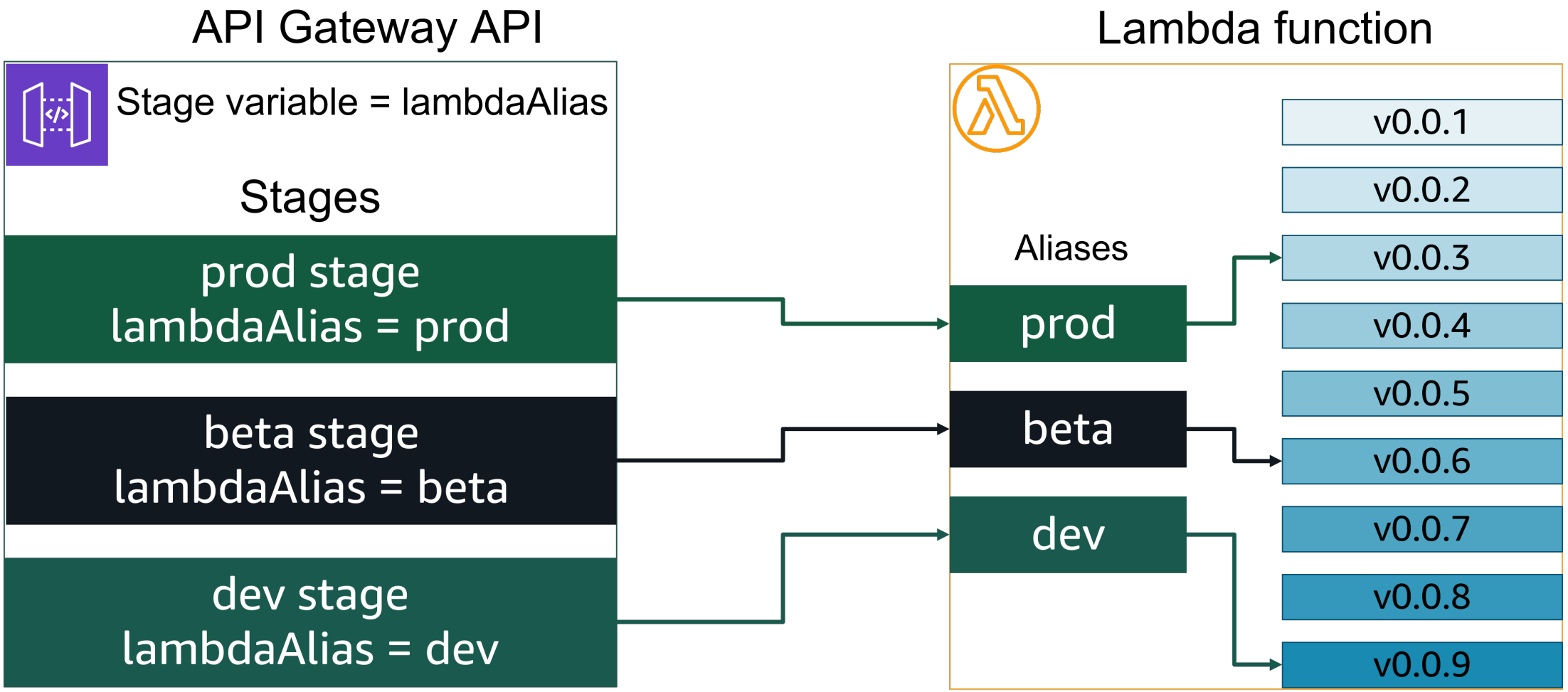
Version 1.0.2: Live to customers



https://api-id.execute-api.us-east-2.amazonaws.com/prod/products

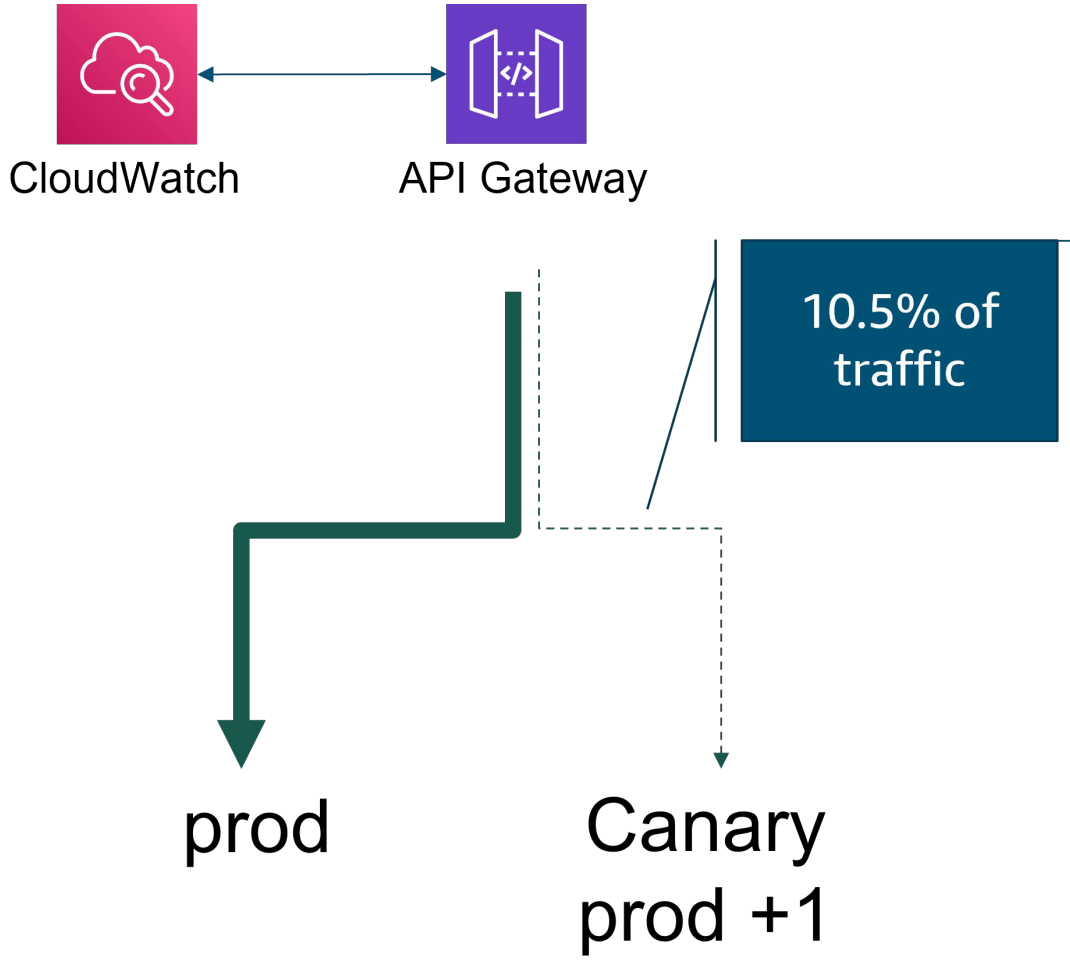
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Connecting to different backends by using stage variables



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

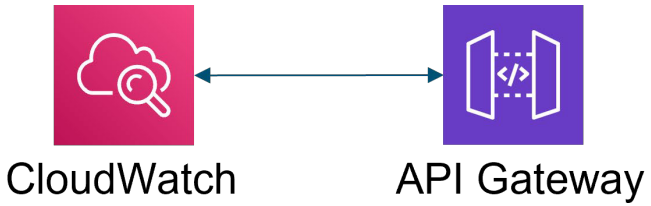
Using canary deployments

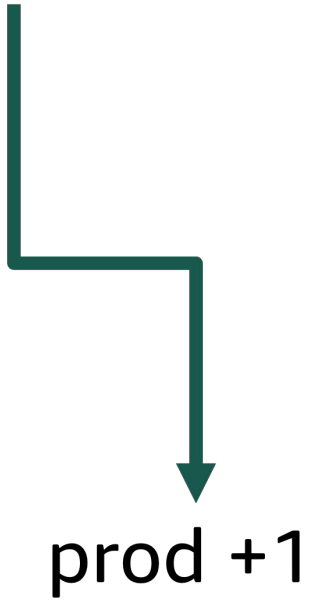



```
aws apigateway create-deployment
    --canary-settings '{ \
        "percentTraffic":10.5, \
        "useStageCache":false, \
        "stageVariableOverrides":{ \
            "sv1":"val2",  \
            "sv2":"val3" \
        } \
}' \
--rest-api-id 4wk1k4onj3 \
--stage-name prod
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Updating canary deployments





prod +1

```
aws apigateway update-stage           \
    --rest-api-id 4wk1k4onj3      \
    --stage-name prod      \
    --patch-operations
op=replace,path=/canarySettings/percentTraffic,
value=25.0
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6 key takeaways

- Stages are snapshots of an API version and are required to deploy your API.
- You can use stages to differentiate deployments for different development environments or backends.
- Use canary deployments to test out an API by sending a percentage of traffic to a new version.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 7: Controlling access to a REST API

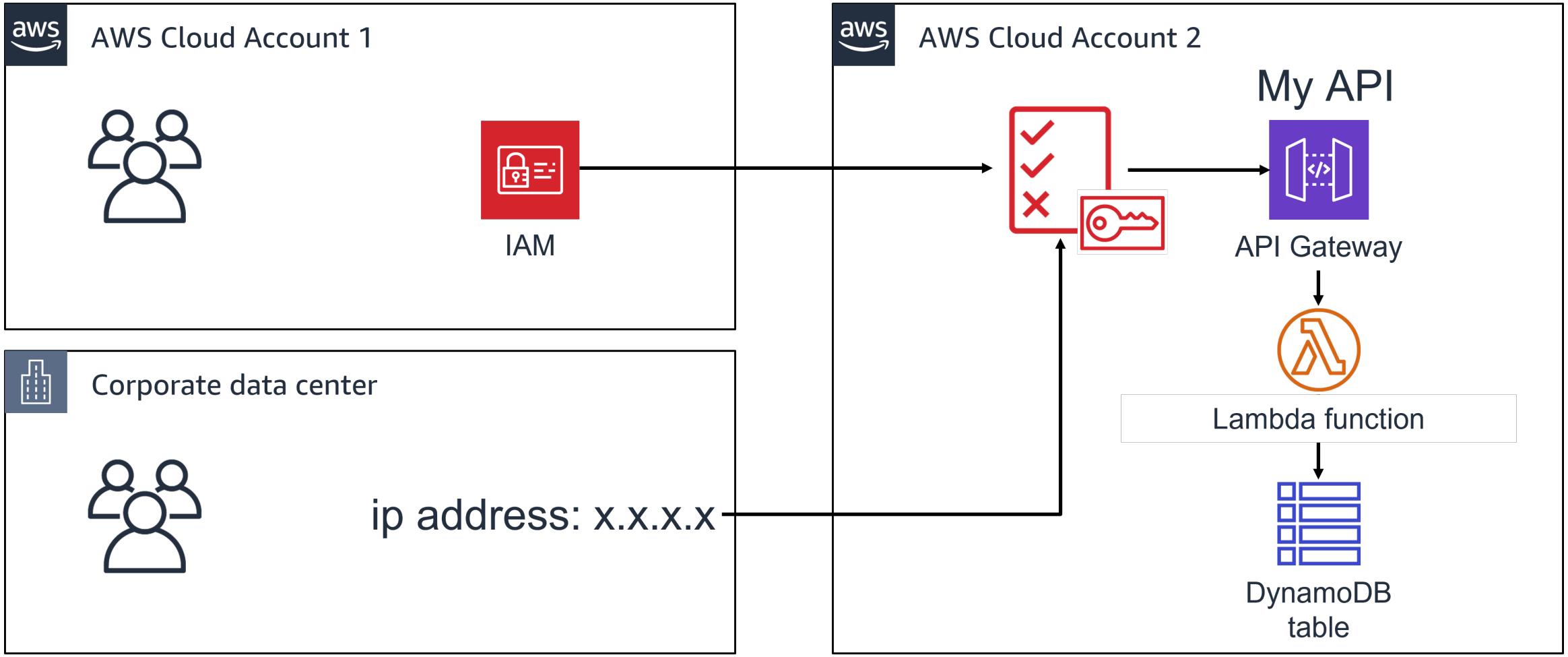
Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Protecting APIs from unwanted traffic

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

IAM resource policies

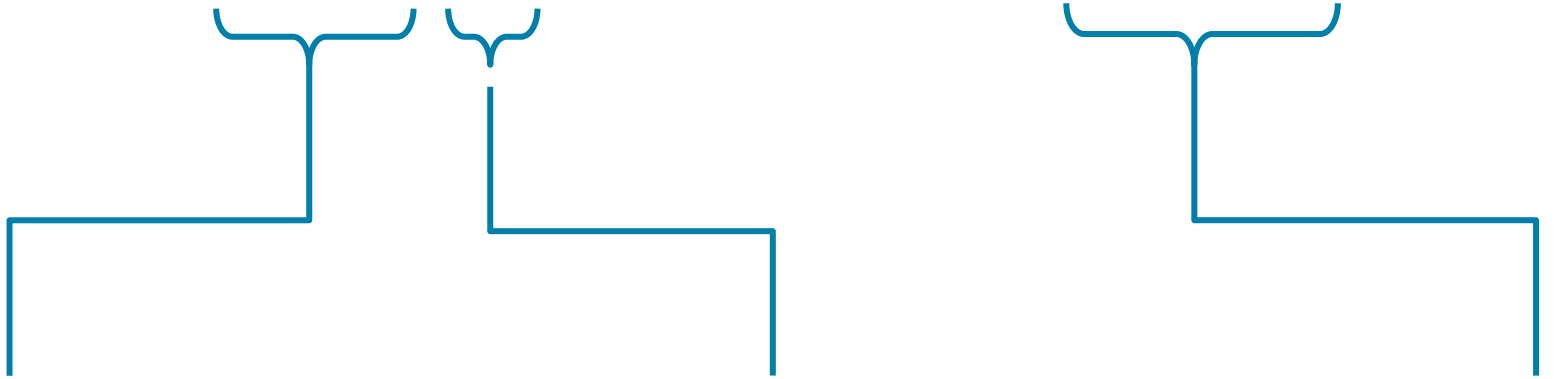


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Custom domains for certificates

https://*12345*.execute-api.*us-east-1*.amazonaws.com/*prod*

https://*.*mydomain.com/api-one*



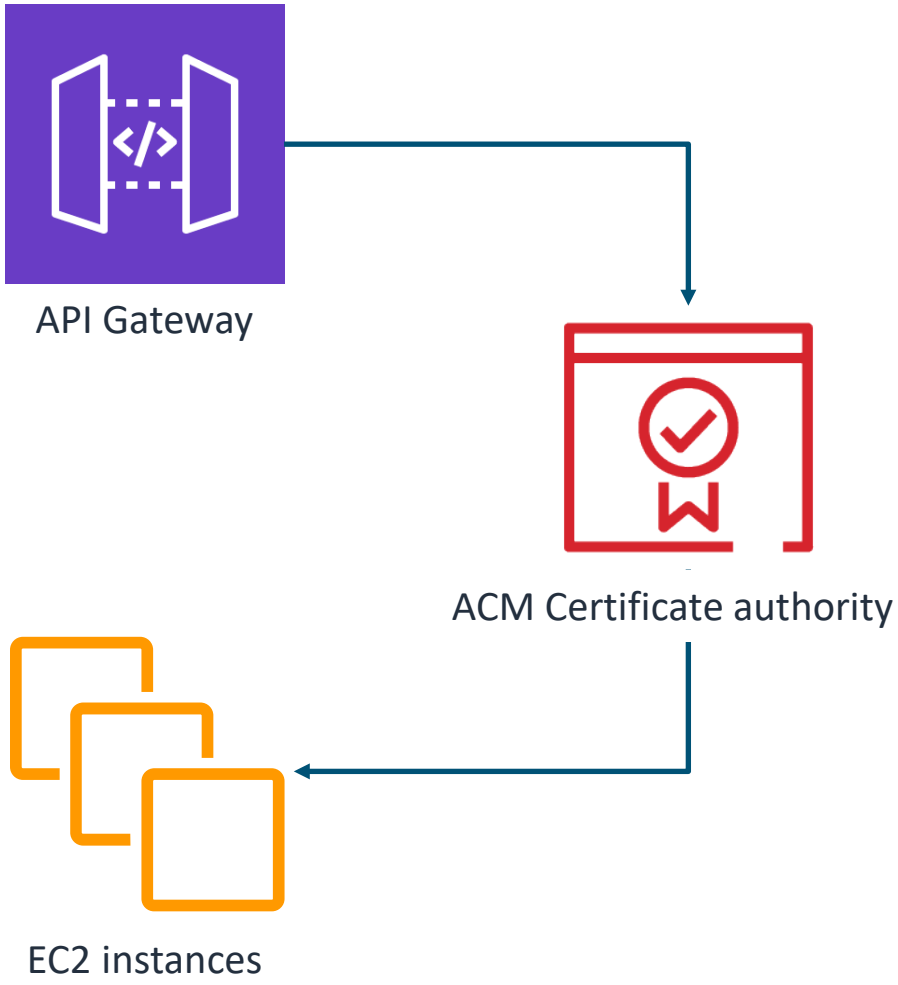
Secure Sockets Layer (SSL) certificates managed through AWS Certificate Manager (ACM)

Supports wildcard (*) domains

Supports multiple domains through base path mapping

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

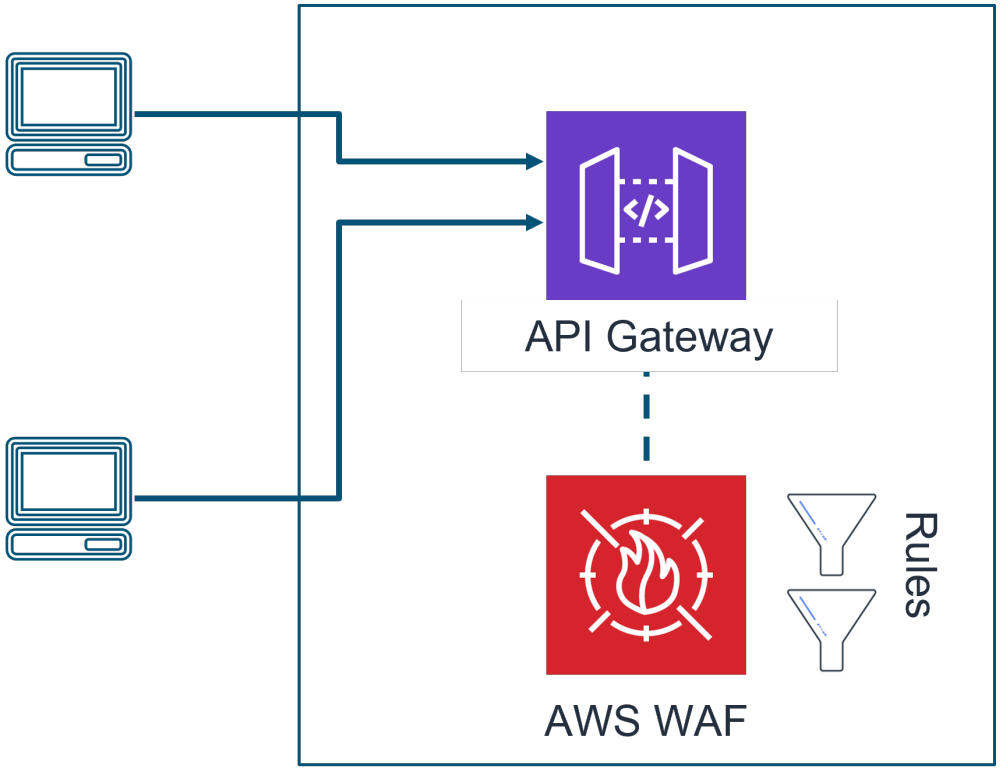
Client certificates



- Generate a client-side SSL certificate by using API Gateway
- The backend verifies the request that comes from API Gateway by using a public key
- Expires after 365 days

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

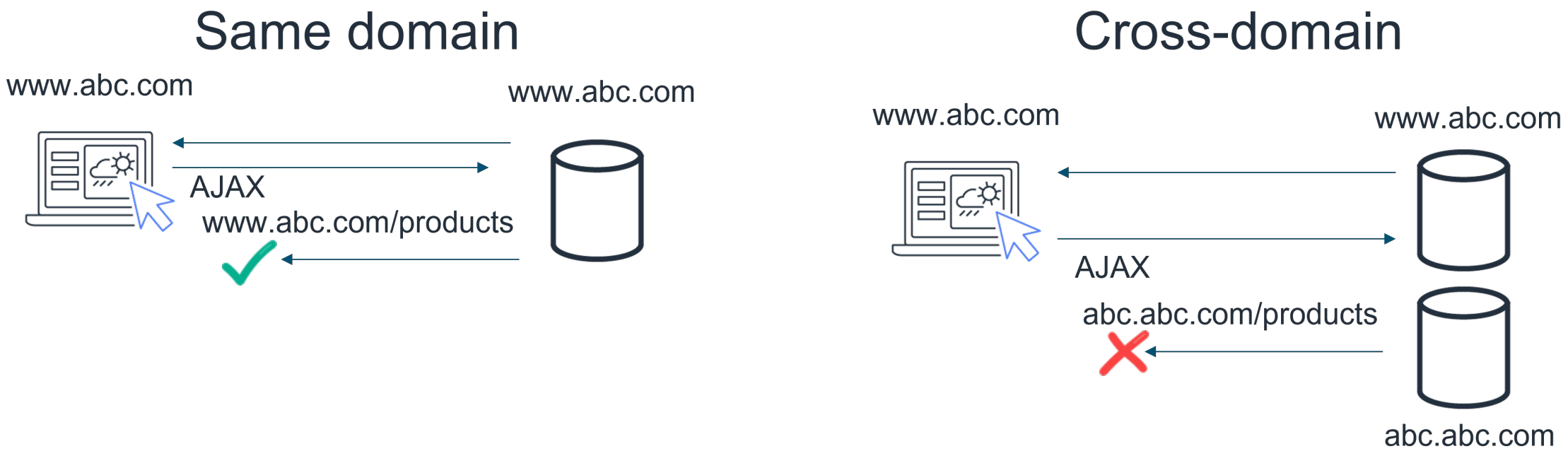
AWS WAF



- Protect APIs from common web exploits like SQL injection and cross-site scripting (XSS)
- Matches a specified string or regular expression patterns
- Blocks requests based on origination from:
  - Specified IP address ranges
  - A specific country or region
  - Specific user agents, bad bots, and content scrapers

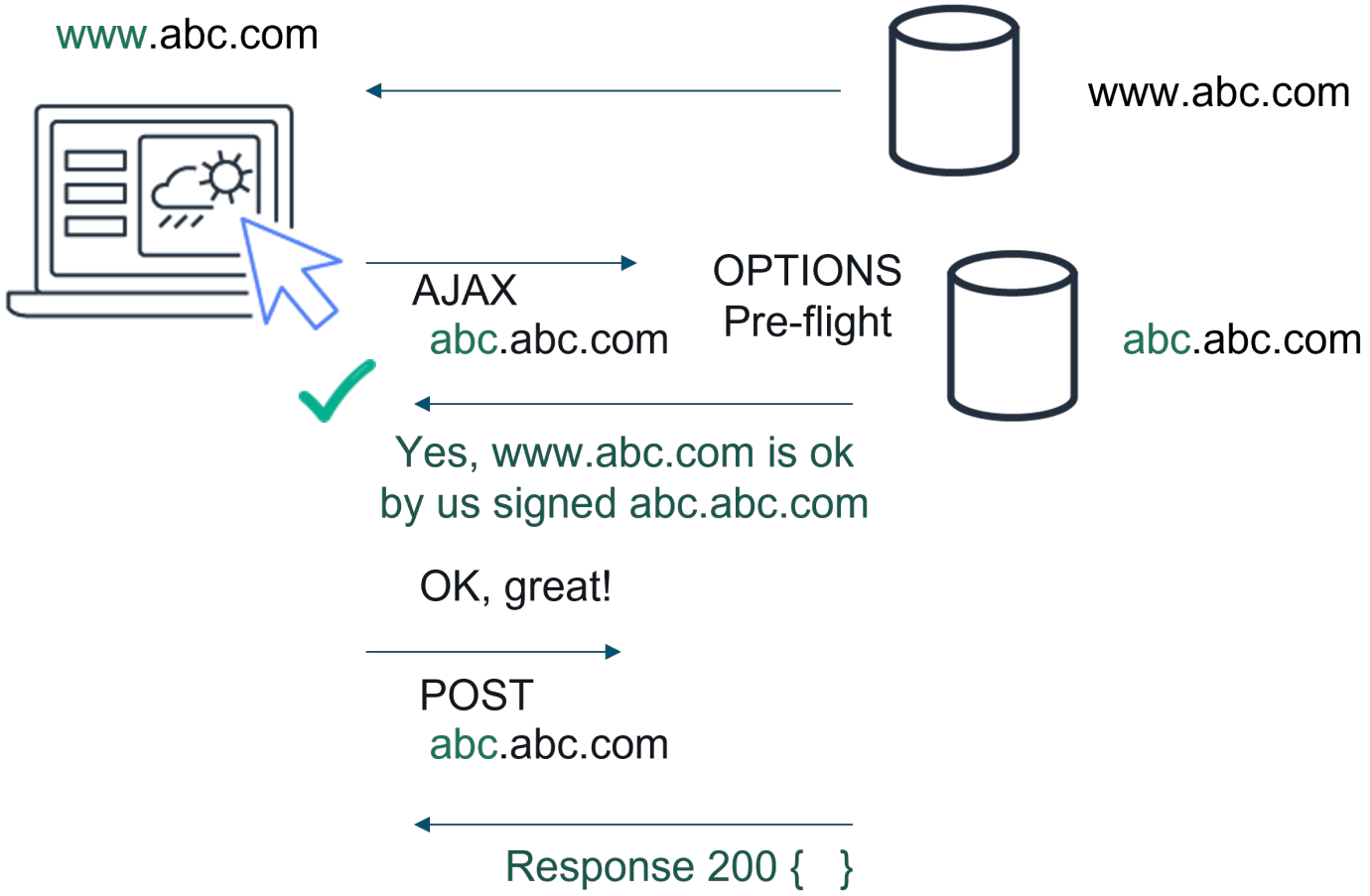
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CORS prevents cross-origin requests



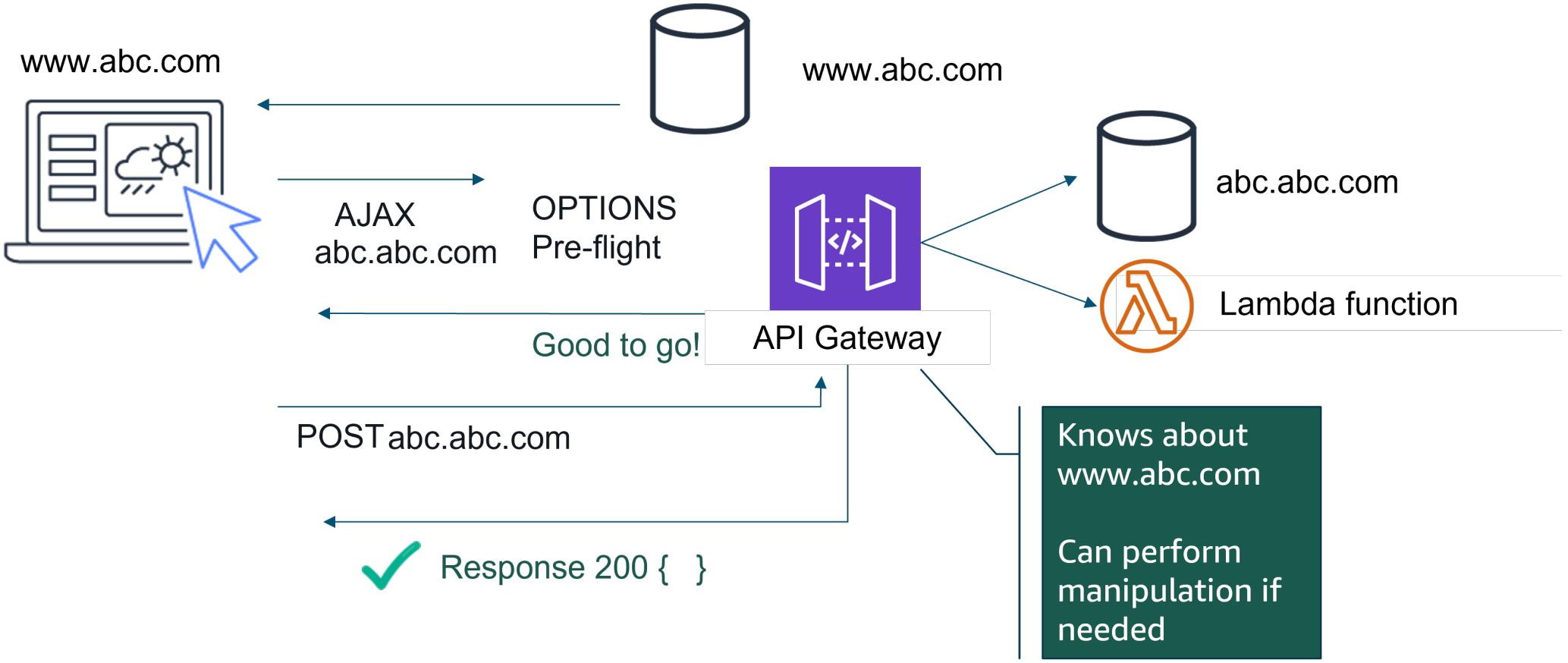
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Cross-origin requests with CORS support



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

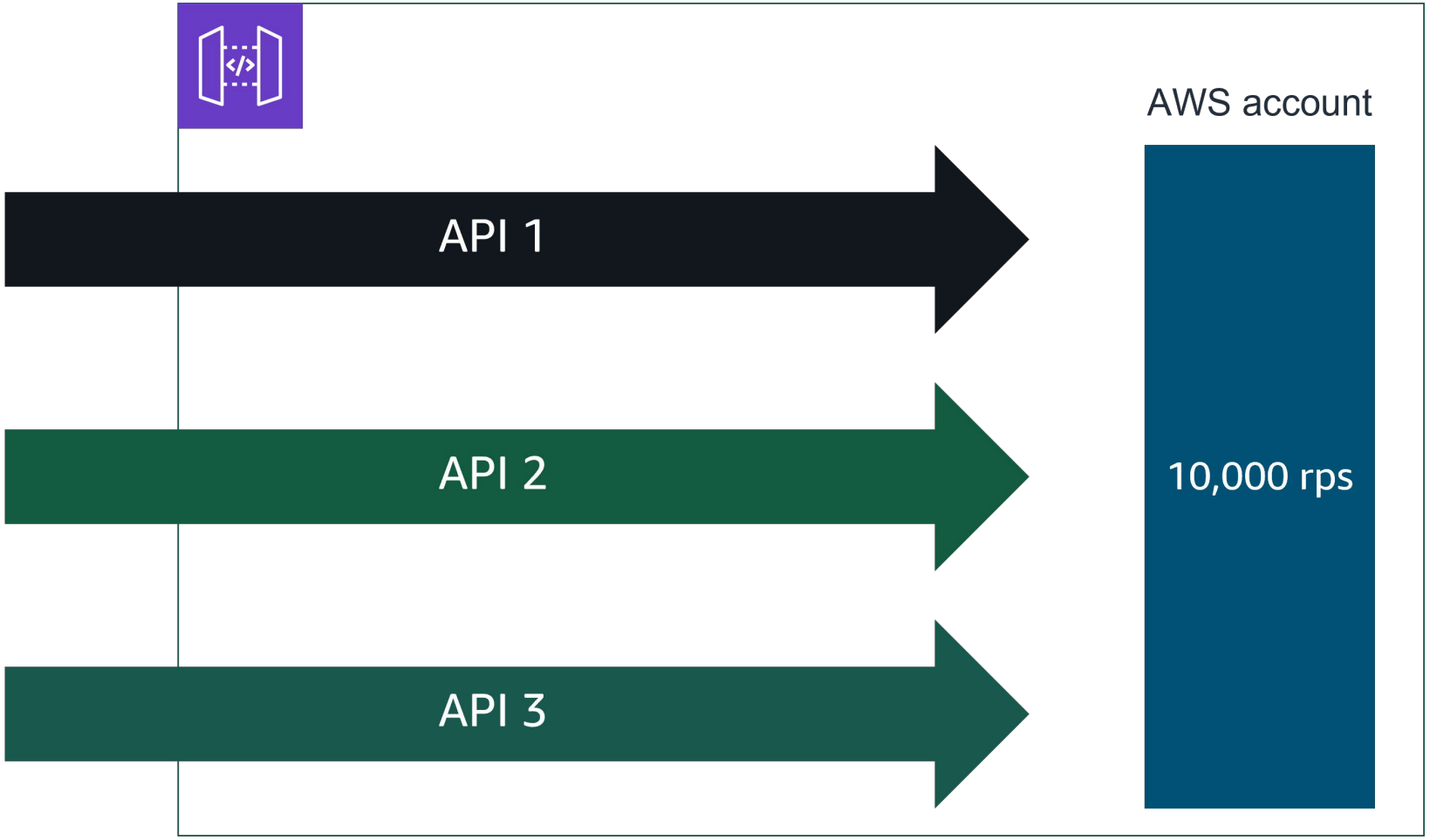
API Gateway with CORS



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Account-level rate limiting (throttling)

All APIs in an account share a Region-wide throttling quota of 10,000* requests per second

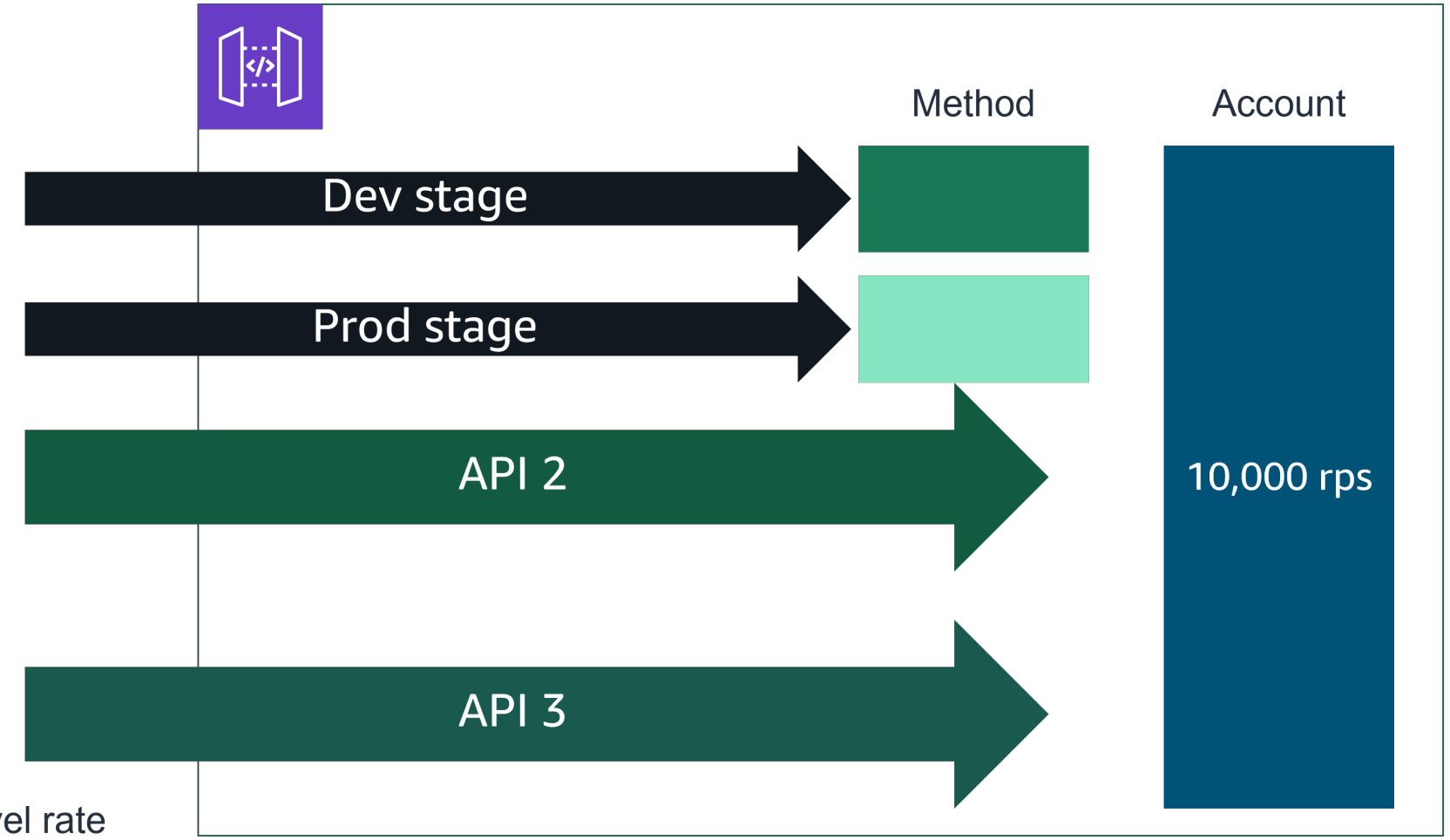


*Soft limit that can be raised on request

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Stage and method or route throttling

Set throttling rates* per stage or per method (REST) or route (HTTP) on your APIs

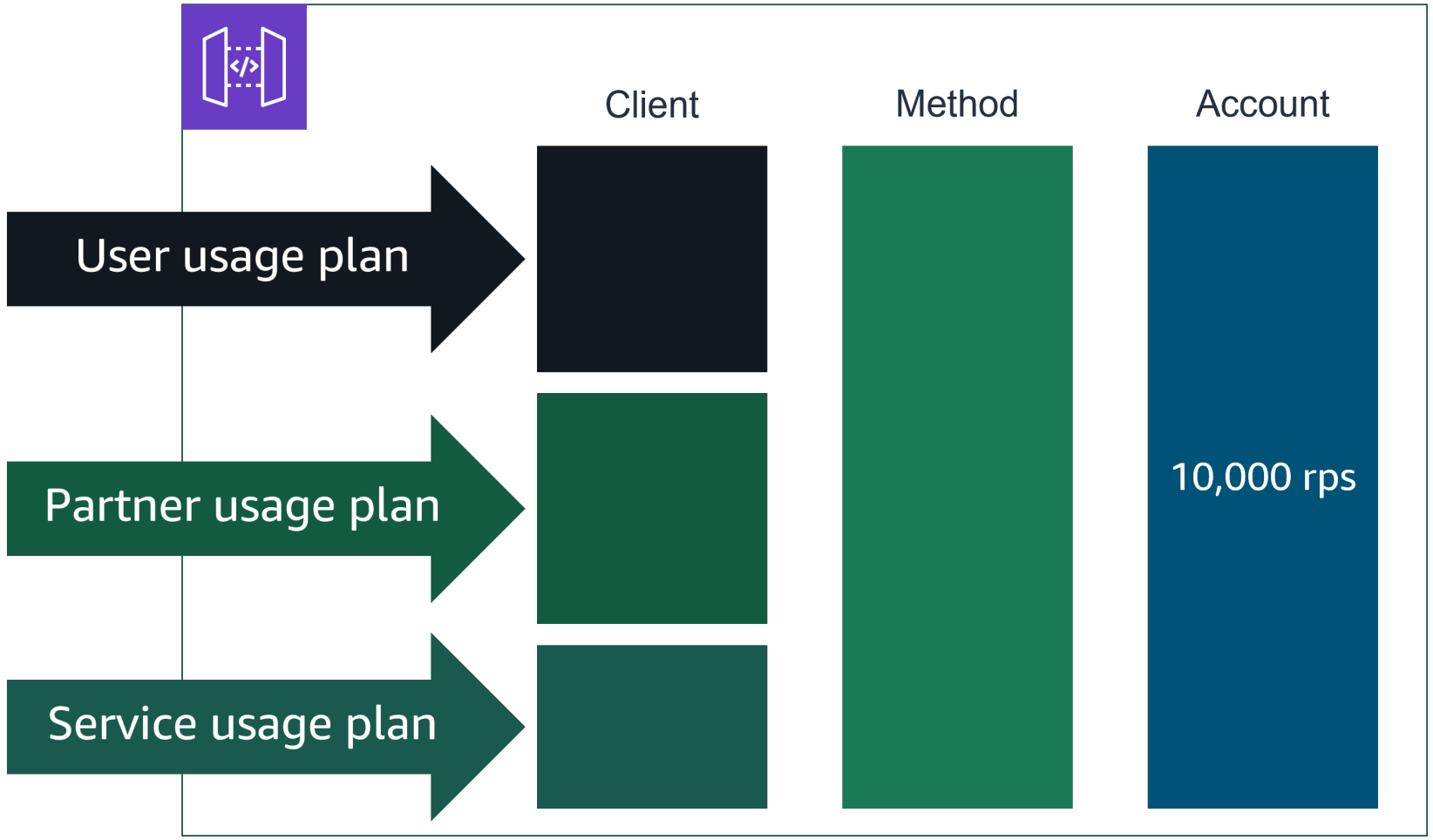


*Cannot be higher than the account-level rate

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Throttling by client with a usage plan

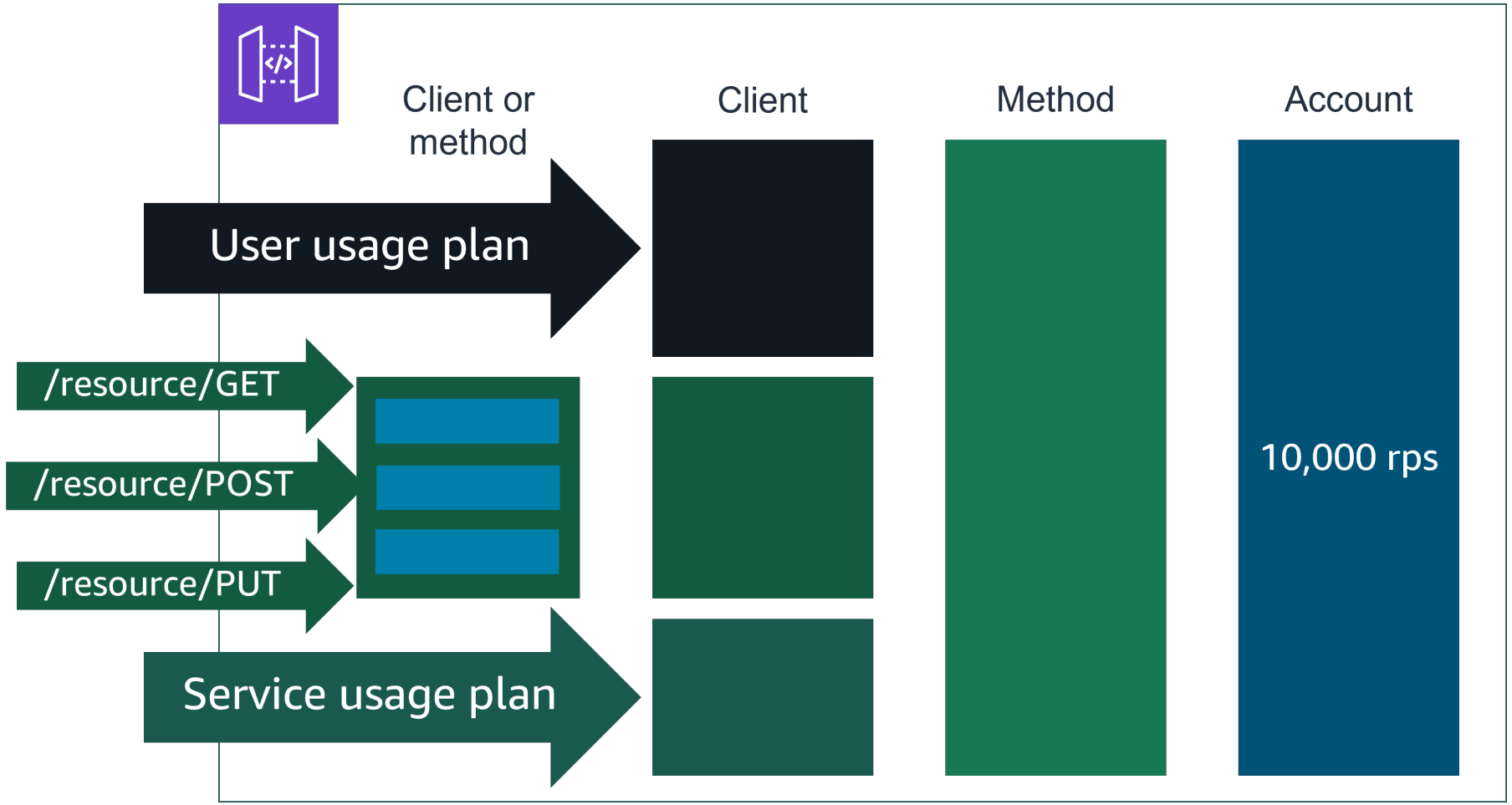
Configure API-level or stage-level throttling by client through a usage plan (REST)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Throttling by client and method

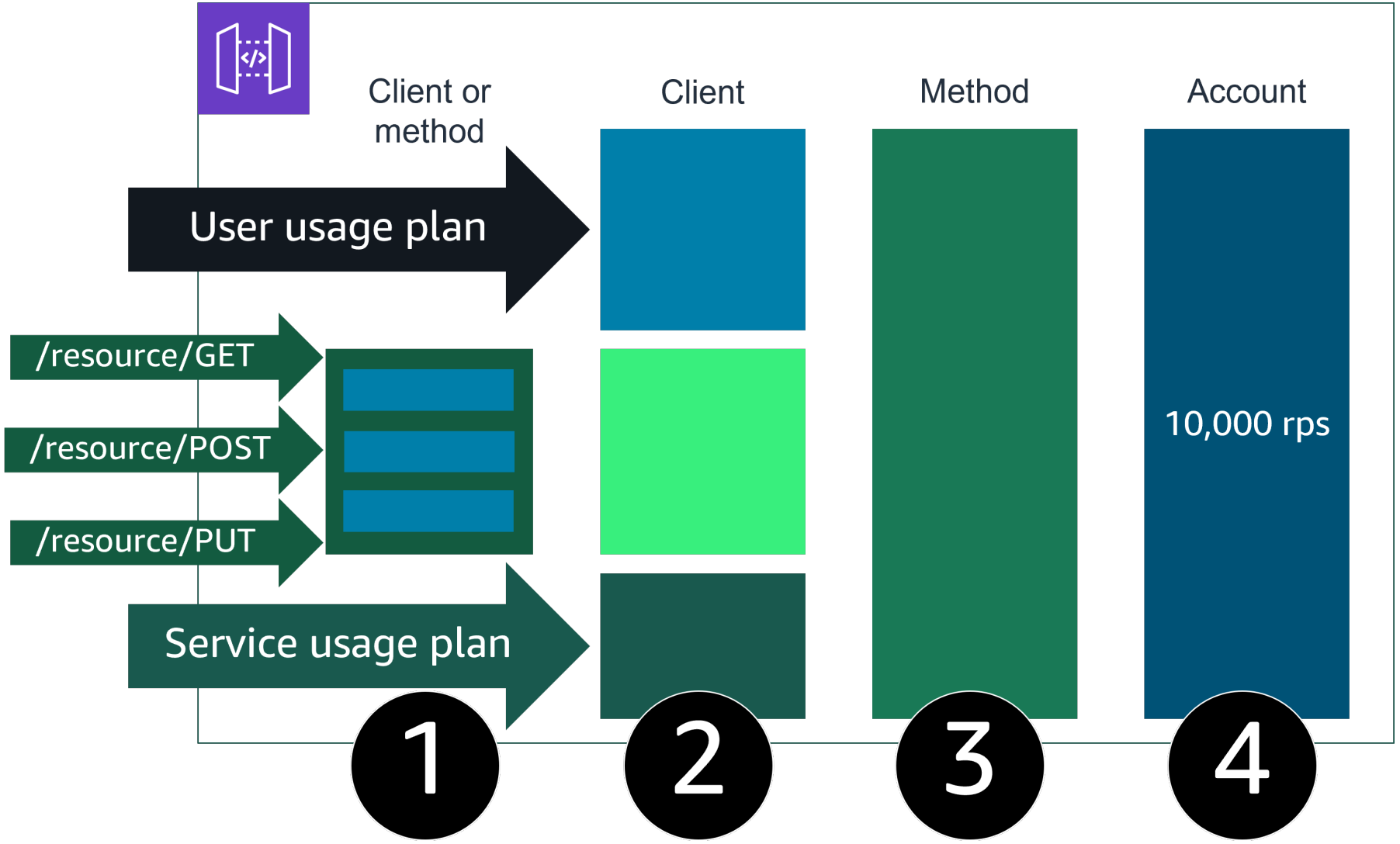
Configure method-level throttling by client in a usage plan (REST)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How API Gateway applies throttling

Throttling settings are applied in a specific order, from most to least granular



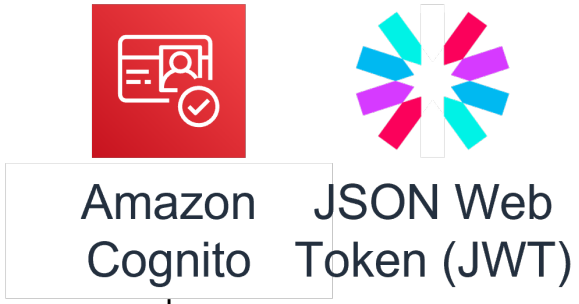
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Authorizing API access

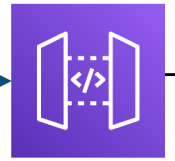
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Controlling authentication and authorization to REST APIs

None
Leave access open



JWT-based
Use Amazon Cognito (REST APIs) or another JWT-based authorizer (HTTP APIs)



API Gateway

Backend

IAM
Use IAM and AWS credentials to grant access

Lambda function

Lambda authorizers
Use a Lambda function to validate a bearer token (like OAuth or Security Assertion Markup Language [SAML]) or request parameters to grant access

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Example: Updating an existing route to use Amazon Cognito user pools

1. Create the authorizer

```
aws apigatewayv2 create-authorizer \
    --name authorizer-name \
    --api-id api-id \
    --authorizer-type JWT \
    --identity-source '$request.header.Authorization' \
    --jwt-configuration Audience=audience,Issuer=https://cognito-
idp.us-east-2.amazonaws.com/userPoolID
```

2. Update the route to use it

```
aws apigatewayv2 update-route \
                --api-id api-id  \
    --route-id route-id  \
    --authorization-type JWT \
    --authorizer-id authorizer-id \
    --authorization-scopes user.id user.email
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lambda authorizers

Function payload without authorization information

```
{
  "context": {
    "exampleKey": "exampleValue"
  }
}
```

Function payload with authorization information

```
{
  "isAuthorized": true/false,
  "context": {
    "exampleKey": "exampleValue"
  }
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 7 key takeaways

- API Gateway supports the following protections from unwanted traffic:
  - Resource policies
  - Client certificates
  - AWS WAF
  - CORS
  - Throttling
- API Gateway supports the following options for authorizing access to resources:
  - IAM
  - JWT-based authorizers
  - Lambda authorizers

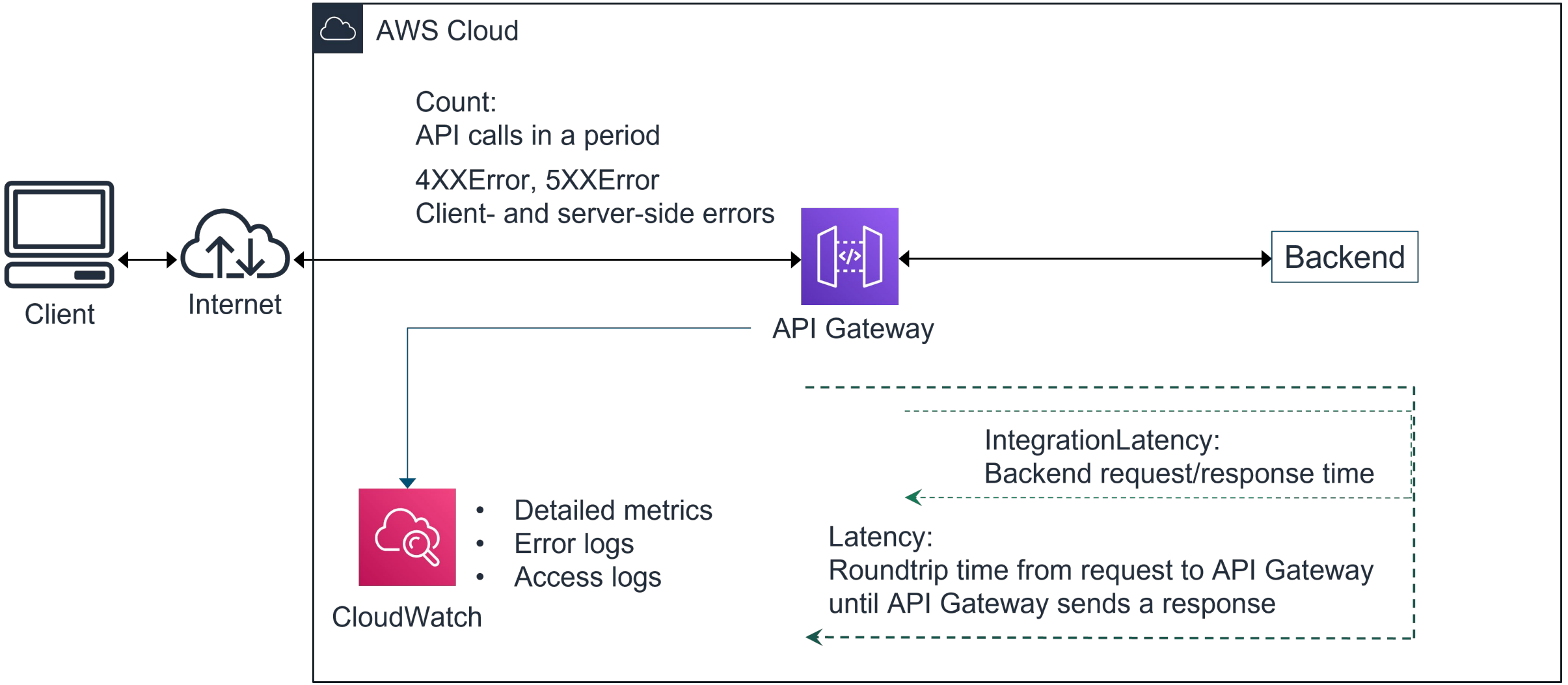
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 8: Monitoring a REST API

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

API Gateway default metrics and logging options for REST APIs



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating access logs

1. Create log group

```
aws logs create-log-group \
        --log-group-name my-log-group
```

2. Set up access logging on a stage

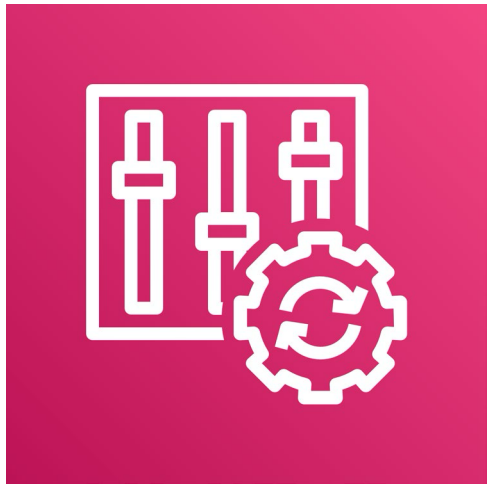
```
aws apigatewayv2 update-stage \
        --api-id abcdef \
    --stage-name '$default' \
    --access-log-settings '{"DestinationArn":
"arn:aws:logs:region:account-id:log-group:my-log-
group", "Format": "$context.identity.sourceIp - -
[$context.requestTime] \"$context.httpMethod
$context.routeKey $context.protocol\"
$context.status $context.responseLength
$context.requestId"}'
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Other monitoring tools

AWS X-Ray

Trace and analyze requests from end to end



AWS Config

Assess, audit, and evaluate configuration updates

AWS CloudTrail

Maintain a record of actions that a user, role, or AWS service takes on API Gateway

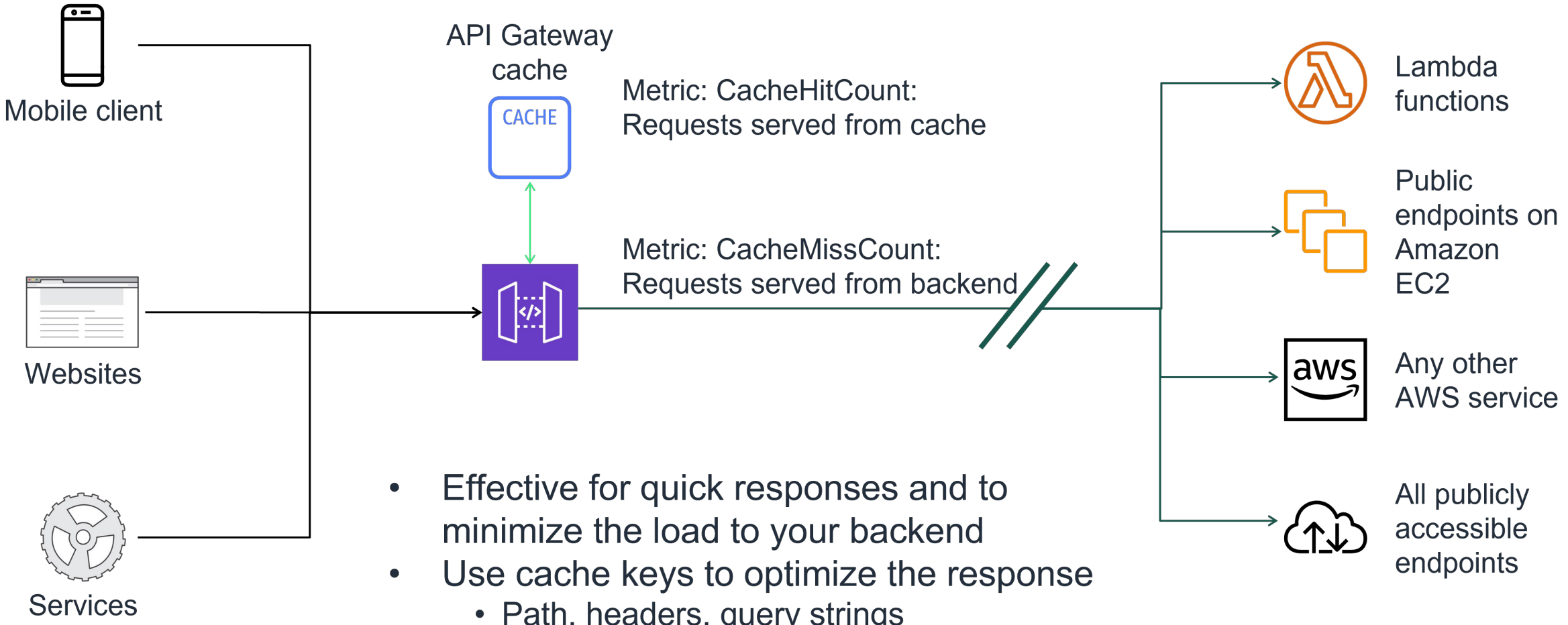
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 9: Optimizing API Gateway

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Caching



- Effective for quick responses and to minimize the load to your backend
- Use cache keys to optimize the response
  - Path, headers, query strings
- Can be set up per stage or method

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Payload compression

Use payload compression to reduce the size of the client payload.
Choose a supported content coding (`deflate`, `gzip`, or `identity`).
Set a `minimumCompressionSize` to tell API Gateway when to apply compression to a payload.
Submit API requests with a compressed payload and an appropriate `Content-Encoding` header.
Test your payload settings to make sure that compression and decompression improve performance.

```
aws apigateway update-rest-api \
    --rest-api-id 1234567890 \
    --patch-operations
op=replace,path=/minimumCompressionSize,value=0
```

Minimum compression value of zero = compress all payloads

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Compression example

Uncompressed client payload

```
{
  "type": "dog",
  "price": 249.99
}
```

Compressed with gzip

```
[illegible]PP*[illegible]HU[illegible]RPJ[illegible]OW[illegible][illegible]&[illegible][illegible],[illegible]-y[illegible]
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab 6.1: Developing REST APIs with Amazon API Gateway

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Scenario

In the previous lab, you played the role of Sofía to build a web application for the café. As part of this process, you created a DynamoDB table named FoodProducts to store information about café menu items. You also loaded data into the table and configured code to extract unique items from the table.

In this lab, you continue to play the role of Sofía. You add the ability to query menu information from the database table through REST API calls. For the initial development, you use mock data endpoints.

In the next lab, you will replace the mock endpoints with real endpoints, which the web application will use to connect to the DynamoDB backend.

aws

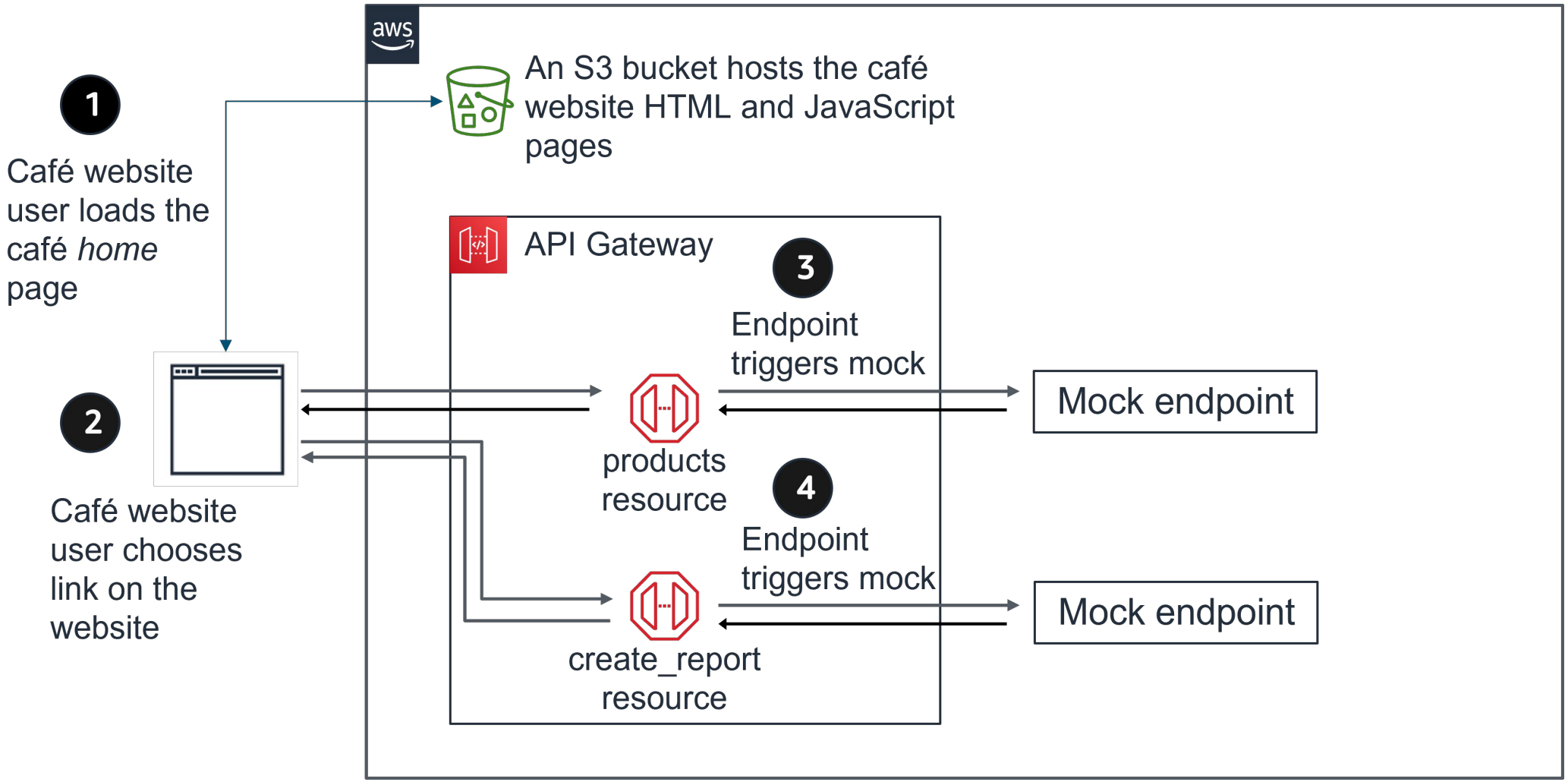
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Tasks

1. Preparing the development environment
2. Creating the first API endpoint (GET)
3. Creating the second API endpoint (GET)
4. Creating the third API endpoint (POST)
5. Deploying the API
6. Updating the website to use the APIs

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

~ 120 minutes

Begin Lab 6.1: Developing REST APIs with Amazon API Gateway

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module wrap-up

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module summary

In summary, in this module, you learned how to do the following:

- Recognize APIs
- Describe API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Complete the knowledge check

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question

A developer is building a web application that uses API Gateway. The developer wants to maintain different environments for development and production (dev and prod) workloads. The API will be backed by a Lambda function with two aliases: one for dev and one for prod.

How can the developer achieve this result with the LEAST amount of configuration?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Different environments**
- **Dev and prod**
- **Lambda function with two aliases**

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Responses

A developer is building a web application that uses API Gateway. The developer wants to maintain **different environments** for development and production (**dev and prod**) workloads. The API will be backed by a **Lambda function with two aliases**: one for dev and one for prod.

How can the developer achieve this result with the LEAST amount of configuration?

Choice	Response
A	Create a REST API for each environment, and integrate the APIs with the corresponding dev and prod aliases of the Lambda function. Then, deploy the two APIs to their respective stages, and access them by using the stage URLs.
B	Create one REST API, and integrate it with the Lambda function by using a stage variable in place of an alias. Then, deploy the API to two different stages—dev and prod—and create a stage variable in each stage with different aliases as the values. Access the API by u\sing the different stage URLs.
C	Create one REST API, integrate it with the dev alias of the Lambda function, and deploy it to a dev environment. Configure a canary release deployment for prod where the canary will integrate with the Lambda prod alias.
D	Create one REST API, integrate it with the prod alias of the Lambda function, and deploy it to a prod environment. Configure a canary release deployment for dev where the canary will integrate with the Lambda dev alias.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Answer

The correct answer is B.

Choice	Response
A	Create a REST API for each environment, and integrate the APIs with the corresponding dev and prod aliases of the Lambda function. Then, deploy the two APIs to their respective stages, and access them by using the stage URLs.
B	**Create one REST API, and integrate it with the Lambda function by using a stage variable in place of an alias. Then, deploy the API to two different stages—dev and prod—and create a stage variable in each stage with different aliases as the values. Access the API by u\sing the different stage URLs.**
C	Create one REST API, integrate it with the dev alias of the Lambda function, and deploy it to a dev environment. Configure a canary release deployment for prod where the canary will integrate with the Lambda prod alias.
D	Create one REST API, integrate it with the prod alias of the Lambda function, and deploy it to a prod environment. Configure a canary release deployment for dev where the canary will integrate with the Lambda dev alias.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Additional resources

- Videos: Serverlessland.com videos about API Gateway
- AWS whitepaper: Security Overview of Amazon API Gateway
- AWS Compute Blog: Announcing HTTP APIs for Amazon API Gateway
- FAQ: Amazon API Gateway FAQs

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.